

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-ac4975ba9e9a4de2a.jsonl`
- SHA-256: `7cf847741e3c13a9c2affa8a1355001701d1cafa7976066632df9a37facb023d`
- Source modified: `2026-07-03T17:39:29+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-03T17:34:26.584Z)

You are a code-review finder agent. Review target: PR #452 of badminton-highlight-indexer at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch codex/a2-golden-manifest-portability (HEAD 5a91701). Diff: git diff origin/master..HEAD. Changed files: backend/eval/golden_manifest.py (new), backend/eval/{calibrate_local,distill_local,fusion_golden,rally_seg_eval,rally_seq_proto,serve_contrast,served_gate_a}.py, training/gen0/harness.py, docs/ALPHA_LAUNCH_READINESS.md, docs/data_pipeline/GOLDEN_VIDEOS.md, tests/test_fusion_compare.py, tests/test_fusion_golden_telemetry.py, tests/test_golden_manifest.py (new), tests/test_served_gate_a_regression.py.

ANGLE ? conventions (CLAUDE.md). Read C:/Users/avidu/.claude/CLAUDE.md (user-level), the repo-root CLAUDE.md / CLAUDE.local.md, and any CLAUDE.md in backend/, backend/eval/, docs/, tests/, training/ (only those that are ancestors of changed files). Also read docs/CODE_MAP.md \$0 file-placement table if referenced by CLAUDE.md, and docs/PROJECT_DOC_TEMPLATE.md \$7 legend conventions for the tracker-row edit (A2 -> 'IN PROGRESS | this PR' ? sibling docs' convention for same-PR rows per the template: check RALLY_DETECTION_FIXES_PLAN.md \$7 rows which use '? DONE | this PR'). Check the diff for CLEAR violations: file placement (does a new eval CLI belong in backend/eval per the CODE_MAP table?), doc honesty tags ([verified]/[analysis] conventions), test conventions, comment/docstring conventions. Only flag when you can QUOTE the exact rule and the exact violating line; name the CLAUDE.md/doc path in the finding. No style preferences.

Return up to 6 candidates as JSON [{file, line, summary, failure_scenario}]. [] if none. Final message = only the JSON array.

2. assistant (2026-07-03T17:34:28.673Z)

I'll analyze this PR against the conventions in your project documentation. Let me start by reading the relevant CLAUDE.md files and documentation.

3. user (2026-07-03T17:34:28.994Z)

```
1      # Global instructions (all projects)
2
3      ## Repo-freshness convention: git pull BEFORE reading
4      **Always `git pull` a repo before starting to read it** ? at session start for the
primary
5      working directory, and again for any sibling/local repo the moment work touches it.
6
7      - **Why:** the owner closes every session on a "clean slate", but multiple parallel
slates
8      (sessions/machines) exist ? PRs get merged and master moves between sessions, so the
local
9      checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff,
docs, code)
10     reflect where the remote actually is.
11     - **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff).
```

```

Never
12     auto-merge, rebase, or discard local state to make a pull succeed.
13     - **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force
anything ?
14         report the state to the owner and proceed read-only on what's local, flagging that it
may be
15         stale. Uncommitted changes are often a sibling session's or the owner's WIP; never
clobber them.
16
17     ## Git branching safety: concurrent sessions / shared checkouts
18     Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and
create
19     branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you
between two
20     commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real
21     incident: a sibling commit landed in the gap between a branch-point check and `git
checkout -b`,
22     so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's
work**.
23     Protocol ? do this EVERY time, not only when you suspect a collision:
24     - **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b
<name> origin/<base>`
25         (e.g. `origin/master`). Never assume the current branch is the intended base.
26     - **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log
--oneline -1`
27         (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must
list ONLY your
28         commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
29     - **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling
or untracked
30         files can't ride into your commit.
31     - **Serialize repo writes:** don't start a repo-writing spawned/background task while
you hold
32         uncommitted work ? commit or push first. If one may be running, treat the tree +
current branch
33         as able to shift, and put this branching-safety reminder into the spawned task's own
prompt.
34
35     ## Session-handoff convention
36     Maintain a living **session handoff** for each project and use it to resume context
quickly
37         across windows.
38
39     - **Where it lives:**
40         - If the project has an auto-memory dir (`~/..claude/projects/<project>/memory/`), keep
the
41         handoff as `session-handoff.md` there, and list it under a **"? Start here"*** entry
at the
42         top of that project's `MEMORY.md`.
43         - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
44     - **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never
duplicates them):
45         1. **You are here** ? current state.
46         2. **Next steps / open threads.**
47         3. **Ramp-up kit** ? the exact files/docs to read to get current.
48         4. **Key decisions** ? so they aren't relitigated.
49     - **At session start:** if a handoff exists, read it before starting substantive work,
and use it
50         (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
51     - **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints
(a PR is
52         pushed/merged, a key decision is made, work shifts phase, or a session is wrapping
up), so a
53         restart can ramp up without losing context. Do **not** rewrite it every turn ? only

```

```

when
54     something worth resuming from changed. The phrases ***update the handoff*** /
***refresh the
55     handoff*** force an immediate full rewrite on demand.
56     - ***Keep it honest:*** it reflects real, shipped state ? never aspirational. (See each
project's
57     attribution/working-agreement note where present.)
58
59     ## Code review ? pre-LGTM gate (applies to every PR/code review, every project)
60
61     Do ***not*** post ***LGTM*** or otherwise sign off on a pull request until I have personally
***verified*** the following ? never trust the PR description's claims, reproduce them locally:
62
63     1. ***Full test suite is green.*** Run the whole suite (e.g. `run_all_tests.py` /
`pytest`), not just the changed files.
64     2. ***Install any dependency needed*** so the suite actually collects and runs ? do not
let modules skip or error for missing deps. (If a check genuinely can't run locally ?
GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
65     3. ***Lint + type checks are clean*** ? `ruff check .` and `mypy ...`, matching the repo's
CI. Install `ruff`/`mypy` if absent.
66     4. ***Coverage stays up*** ? run with coverage and confirm it stays ***at or above the
repo's floor*** (e.g. the CI `--cov-fail-under` value) and does ***not regress*** versus the base
branch.
67     5. ***Review findings are resolved*** ? bugs and repo-convention/discipline issues
addressed.
68
69     LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't
verifiable locally.
70

```

4. user (2026-07-03T17:34:29.149Z)

```

1     # CLAUDE.md ? project memory / agent handoff
2
3     Local, AI-powered ***badminton (? racket-sports) highlight indexer + rally detector***:
4     FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid ***Gemini*** as the cloud AI. Sibling
5     repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7     ## Read these first
8     - ***docs/README.md*** ? doc index. ***docs/CODE_MAP.md*** ? code navigation + data
9     contracts (the cold-start map; read before touching code).
10    - ***Authoritative owned-model roadmap:*** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11    live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12    disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13    - ***Strategy (post-scaffolding):*** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14    `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15    (offline-segmenter narrative) and `docs/BACKLOG.md`.
16    - ***Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):***
17    `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18    (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19    "Gate A" ? implemented in `backend/eval/served_gate_a.py`, default-OFF) ?
`docs/EXPERIMENT_HARNESS.md` (A/B + shadow
20    eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21    (v8 vendor labeling spec). ***docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.***
22    - ***History:*** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23    ***checkpoints/*** (latest: `2026-06-strategy-foundation/`). ***Archive policy:*** only

```

```

move
24     **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25     archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
26     note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27     lifecycle register in **`docs/DOC_STATUS.md`**.
28
29     ## Current state (July 2026)
30     - **Platform slice SHIPPED:** async job model (`backend/api/job_worker.py`, #339),
31       `StorageBackend` + `ComputeBackend` registries with concrete implementations
32       (`backend/storage/` ? local/gcs/gdrive/gdrive_api/s3; `backend/compute/` ?
33       local/cloud_run), capabilities/assets endpoints, and the data flywheel
34       (`backend/flywheel.py`). `COMPUTE_DECOUPLED_SERVING` M0?M9 largely shipped
35       (ADR-014 ratified 2026-06-21); Gen-0 owned-model weights pushed (`weights/0.1.0-l4/`).
36     - **Launch decision: NO-GO (owner, 2026-06-30, #404)** ? the rally-detection product is
37       NOT cleared for external/commercial launch; we stay in development. Two unblockers
gate a
38     future GO: (1) set + meet the ship-gate target (#172, needs corpus growth); (2)
resolve
39     WASB shuttle-detector weight provenance (commercial-clean licensing).
40     - **Active tracks** (see `docs/NEXT_STEPS.md` for the prioritized pending list):
41       rally-quality / owned-model ML work (gated on golden-corpus growth), packaging
42       (`docs/PACKAGING_PLAN.md`), and the khelsutra SPA/deploy sibling track.
43
44     ## Architecture in one breath
45     - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
46       / sports / annotations ? see `backend/pipeline/segmenters/base.py`.
47       `StorageBackend` + `ComputeBackend` registries are **implemented**
48       (`backend/storage/`,
49       `backend/compute/`; PLATFORM S3 ? realized by `COMPUTE_DECOUPLED_SERVING`).
49     - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
50     - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
51       user_version` migration ladder in `backend/infrastructure/database.py`.
52
53     ## Committed guardrails (non-negotiable)
54     - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
55       training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
56     code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
57     - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
58     is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
59     needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
60     minors** from the training pool; per-user training data needs a separate explicit opt-
in.
61     - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
62       (GDPR Art. 9).
63     - **Generic data schema** ; `coach ? athlete` is an **optional overlay**, not baked in.
64     - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
65       (`deploy/gcp/`;
66       Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
67       on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
68       DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
69       `deploy/digitalocean/` scripts are kept solely as provider-agnostic Linux-GPU setup

```

reused by GCP.

```
69
70     ## Workflow conventions
71     - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
72     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
73     - Never push to the default branch directly; descriptive commit messages.
74     - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
75     - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
76     - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
77     who has the golden corpus + labelled videos; verify pure-logic paths locally.
78
```

5. user (2026-07-03T17:34:29.592Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-142 of 390 total (40767 tokens, cap 25000). Call Read with offset=143 limit=142 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```
1     # CODE_MAP.md
2
3     **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
4
5     **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure
(this revision follows #40).
6
7     ? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust
git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no
FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via
`segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not
basename; **indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec`
defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**;
```

`match\_intervals` requires real overlap (iou>0); `run\_regression` baseline lives in tracked
`eval\_baselines/` and **exits 3 on a missing baseline**;

the API validates
`video\_path`/`output\_filename`. New tests: `test\_api\_endpoints`, `test\_compiler`,
`test\_wasb\_runner`, `test\_reingest\_safety`, `test\_detector\_keying`, `test\_config\_wasb`,
`test\_path\_safety`, `test\_reliability\_hardening` (+ collector `test\_licensing\_gate`,
`test\_import\_guard`, `test\_cvat\_robustness`).

```
8
9     **LEGEND**
10    - `@path` = file (always repo-relative)
11    - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
12    - `->` = dataflow / call / "produces"
13    - `$` = data contract (canonical shape)
14    - `?` = gotcha / footgun
15    - `?` = registry / plugin extension point
16    - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
17
18    ---
19
20    ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)
```

```

21
22     Top-level tree and the placement rule for each kind of file. When unsure, prefer an
existing
23     registry / extension point (`?`) over a new top-level file.
24
25     | You're adding? | Put it in? | Notes |
26     |---|---|---|
27     | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are register a class in the relevant `? registry, not a new module tree. |
28     | Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are imported widely (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29     | Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app
code) | scripts/` | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30     | Ops / maintenance tool (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31     | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32     | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a 70%
coverage floor. Pure/offline + mocked (no GPU/WSL/network). |
33     | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
terminal-state (DONE/REJECTED) work moves to `docs/archives/`. |
34     | A throwaway repro / debug script | `scratch/` | Gitignored + excluded from
ruff/pytest. Never imported by app code. |
35     | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
manifest committed (provenance), weights gitignored. |
36     | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37     | Committed regression fixture (video-free golden) |
`eval_baselines/fixtures/<slug>/` | Tracked, LF-pinned (`gitattributes`). Synthetic Tier-0
(or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with
no video/GPU/DB. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m
backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
38     | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | Gitignored.
Per-video isolation is the tracked PER_VIDEO_WORKSPACE.md project (P0 helper shipped #264;
supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |
39
40     Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do
not move):
41     `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
42     `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
43     (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44     `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45     New standalone run-drivers go in scripts/`, not the root.
46
47     ---
48
49     ## 1. PIPELINE
50
51     ### 1A. Runtime: video file -> highlight reel
52     ``
53     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54
55     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig

```

```

55         -> video_id = MD5(whole file)                @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56         -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57         FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
58         [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59         -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
60         -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
61         -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
62         -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
63         [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
64         -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
65         -> select segment ids -> [(start,end)] (+ stitching padding)
66         -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
67         per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
68         ...
69         Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):
70         ...
71         P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
72         -> P2 candidate "action windows" from motion/trajectory [STOP: candidates persisted
via db.add_candidate_segment]
73         -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor ->
provider.analyze_video(clip, prompt) ? provider_registry
74         -> P4 db.add_play_segment + db.link_candidate_to_segment
75         ...
76         Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
77         Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground
truth).
78
79         ### 1B. WASB shuttle substrate (P2 of wasb_hybrid)
80         ...
81         WasbHybridSegmenter.process_video
@backend/pipeline/segmenters/wasb_hybrid.py
82         -> WasbRunner.healthcheck() (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
83         -> WasbRunner.run_predict(video, out_dir)
84         stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
85         -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU
detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
86         -> TrackNetRunner.parse_trajectory_csv(csv)
@backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
87         -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING
BRAIN]
88         ...
89         (`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width`
fallback (30.0, 1280.0).)
90
91         ### 1C. Calibration / eval flow (NO pipeline run unless told)
92         ...
93         GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts ->

```

```

List[Interval]
94     DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]    stage
? {candidates, final}
95     -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
96     -> harness.evaluate -> EvalReport -> report_to_dict
97         -> batch.aggregate (corpus micro/macro)                                @backend/eval/batch.py
98         -> regression.regress(_with_provider) vs baseline    @backend/eval/regression.py
[CI gate]
99     Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate}    @backend/eval/calibration.py
100     Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned
classifier)    @backend/eval/{calibrate_local,distill_local}.py
101     Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
102     Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies}    @backend/eval/gemini_refine.py
103     Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
104     ...
105     Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@scripts/nightly_regression.py` (nightly CACHED-trajectory CPU rally-quality tripwire, exit
1=regression/4=stale), `@scripts/nightly_reelcount.py` (nightly FULL-pipeline reel-count +
quality + cost on a GCP GPU VM, emailed), `@backend/eval/calibrate_wasb.py` +
`@backend/eval/export_wasb_segments.py` (WASB tuning drivers).
106
107     ---
108
109     ## 2. SUBSYSTEMS
110
111     ### 2.0 Orchestration & config
112     - `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS).
Static UI mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig`
(INFO, timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces
stray prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI
summaries may still use `print` for direct output.
113     - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module
globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` ->
every endpoint NPEs.
114     - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+ job_id**. `::drain_due_jobs` / `::run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`
`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs
`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the
pipeline synchronously inline (separate code path, unchanged by #16).
115     - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in
`@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`;
`uploads.py::upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the
startup config, and a lazy `import backend.main` avoids an import cycle) ?
`uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext
allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-
bytes body, STRICTLY sequential n, streams to `partial-<id>` under `security.upload_dir`; per-
part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete`

```



```

({total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ?
client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ?
upload state (`uploads.py:::uploads` + `:::uploads_lock`) is IN-PROCESS ? restart aborts
partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight`
`@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-
name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert
path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads',
max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge
proxies cap bodies ~100MB).
116     - **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities`
`@GET /api/capabilities` ? honest box probe; now includes a **`storage` block**
(`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive
`{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment.
`::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET
/api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy**
(D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes
the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset`
`@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a
video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-
in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in
`/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-
auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`,
`::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):**
`@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` +
the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS**
(`detectors/base.py::wsl_frame_stage_shortfall`, ~15x source).
117     - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly.
`::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest`
pydantic bodies.
118     - `main()` resolves `global_config.output.output_dir` through the app-home-aware
resolver; `--output-dir` overrides it only when the CLI flag is explicitly passed (otherwise
`config.json` wins). `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
119     - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
120     - `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive
`input()` (blocks automation), skips validation. Config via `backend.config.load_config`
(typed); segmenter, padding, and db path all read from the model (no literal fallbacks).
121     - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/I/O. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
122     - `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).
123     - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
124     - `@scripts/nightly_regression.py::main` ? **nightly rally-quality tripwire** (the dual-
eval-set discipline made automatic). Re-scores the golden corpus
(`output/human_lovo_manifest.json`) under TWO configs ? `default` (SERVED, milestone knobs OFF)
+ `milestone` (R3 cap=6 + R4 inactivity trim) ? via `backend.eval.rally_seg_eval.evaluate` (CPU,
seconds; CACHED trajectories, **no GPU/Gemini/WASB**), diffs headline metrics (`tol_f1`, tIoU
`f1@0.5/0.25`, `merge_rate`) + the **per-video over-seg guard** (`split_count`/`merge_count` may
not rise on ANY video) vs frozen `eval_baselines/nightly_baseline.json`. Pure core
(`summarize_run`/`compare_config`/`compare_all`/`format_report`) is unit-tested; thin I/O shell
scores + writes `output/nightly_regression/<date>.{md,json}`. Exit **0=pass, 1=regression,
2=missing corpus, 3=no baseline, 4=stale (corpus/config changed ? re-snapshot)**. `--update-
baseline` snapshots HEAD. **Default-ADDITIVE ? does NOT change the promotion gate.** Wired as a

```

```

local Windows Scheduled Task via `scripts/nightly_regression.ps1` (wrapper) +
`scripts/register_nightly_task.ps1` (a cloud agent can't ? the cached trajectories live outside
the repo). The future **RAW eval set** (currently-bad videos) plugs in as a third config. See
`docs/R2_EVAL_METRIC_DESIGN.md` + `memory/eval-dual-set-regression`.
125 - `@scripts/nightly_reelcount.py::main` ? **nightly FULL-pipeline engine regression**
(the GCP-VM sibling of nightly_regression; runs the REAL configured+checkpointed WASB pipeline,
not a cached re-score). Per manifest video (`eval_baselines/reelcount_manifest.json`, gs://
clips): `segmenter_registry.get(seg)(db,cfg).process_video` ? **reel count** =
`len(play_segments)` + (if labels) R2 quality (`rally_seg_eval.score_intervals`); diffs vs
`eval_baselines/reelcount_baseline.json` ? reel-count **EXACT** (`random.seed(0)` + re-run-once
guard for the transient `add_play_segment`?None flake), quality with tolerance; **cost** = VM-
uptime × $/hr (`backend.billing`, USD+INR). Pure core
(`summarize_results`/`compare`/`cost_report`/`format_report`) is unit-tested
(`tests/test_nightly_reelcount.py`); `scripts/nightly_email.py` emails the report (SMTP creds
from Secret Manager/env, graceful no-creds skip). Exit **0/1/2/3/4** (mirrors
nightly_regression). Runs on an ephemeral GCP GPU VM that self-deletes ?
`deploy/gcp/nightly_regression/` (`run_on_vm.sh` startup-script, `launch.sh` driver-agnostic,
`register_scheduler.sh` Cloud Scheduler?Cloud Run job). GPU/email paths owner-verified (not in
CI). See `deploy/gcp/nightly_regression/README.md` + `docs/NIGHTLY_ENGINE_REGRESSION.md`.
126
127 - `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`StitchingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly those 7. Use `from
backend.config import load_config, AppConfig`.
128 - `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for
defaults**.
129 - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config` = {extra:'allow', validate_assignment:True}. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
130 - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain
dicts** (`model_dump(mode='json')`, NOT the model) so chained
`config.get("indexing",{}).get(...)` keeps working; falls back to searching a leaf key in any
section (sections discovered dynamically from `model_fields`). ? "bit every validator on real
runs" per docstring ? returning the model breaks `.get`. `::AppConfig.__getitem__` raises
`KeyError` for unknown top-level keys else delegates to `.get`.
131 - `::IndexingConfig` ? segmenter/detector knobs (`motion_threshold=0.08`,
`min_segment_duration=3.0`, `dead_time_merge_gap=2.0`, `chunk_duration=90.0`,
`chunk_overlap=10.0`, `yolo_velocity_threshold=0.15`, `yolo_frame_skip=3`,
`ai_handoff_padding=2.0`, `ai_max_workers=5`, `log/store_yolo_candidates=True`, nested
`tracknet`). ? `default_segmenter='yolo_hybrid'` HERE but config.json overrides to `gemini`. ?
`::IndexingConfig.segmenter_must_be_registered` `@field_validator` is a **no-op pass-through** ?
a typo'd segmenter validates fine, only fails later at `segmenter_registry.get(...)` .
132 - `::ValidationConfig` (`min_resolution_width=1280`, `min_resolution_height=720`, `min_f
ps=29.9`, `min_blur_threshold=20.0`, `max_scene_cuts_per_minute=0.5`, nested `relevance`);
`::ValidationRelevanceConfig` (court HSV gates, `court_pixels_min_ratio=0.05`).
`::TrackNetConfig` (WSL invocation contract: `wsl_distro='Ubuntu'`, `repo_dir`,
`conda_env='TrackNetV4'`, `weights_path='', `queue_length=5`, `velocity_threshold=0.02`).
`::OutputConfig` (`default_highlights_name`, `db_name='sports_indexer.db'`,
`output_dir='output'` ? CLI `--output-dir` overrides it on the typed model in `main()`).
`::StitchingConfig`
(`begin_padding=0.5`, `end_padding=0.5`, `use_cache=False`, `min_shot_count=1`). `::Sport` Literal
of 5 sports. `::Config` = `AppConfig` alias.
133 - `@backend/config/loader.py` ? load/save with defaults-merge.
134 - config path = `backend.utils.app_home.default_config_path()` (P0a; honours
`RALLY_APP_HOME`, else `Path("config.json")` CWD-relative ? the old `DEFAULT_CONFIG_PATH`
constant was dropped as a frozen-at-import footgun). `::get_builtin_defaults` =
`AppConfig().model_dump(...)` .
135 - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-
level merge** (`{**defaults, **file_data}`): a section in config.json REPLACES the whole
defaults dict for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
136 - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present
(used by scripts/doctor.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.

```

```

137     - `@config.json` ? on-disk config; mirrors `AppConfig`. ?
138     - `@scripts/doctor.py` (formerly root `setup.py`) ? `::init_default_config` ? delegates
139     - `@backend/config.save_default_config` (single-source defaults; skips if config.json exists).
140     - `::run_pip_install` GPU-aware (`nvidia-smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8,
141     - `ffmpeg/ffprobe` on PATH, optional torch+CUDA). Run via `python scripts/doctor.py` or `indexer
142     - `--setup`.
143
144     ### 2.1 Segmenters (under pipeline/segmenters) ?
145     - `@backend/pipeline/segmenters/base.py`
146     - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
147     - `name`, `description`, `process_video`. $ `process_video(video_id, video_path, player_pool=None,
148     - `max_frames=None)` -> List[dict{id,start_time,end_time,duration,metadata}]`.

```

6. assistant (2026-07-03T17:34:32.139Z)

Let me continue reading the CODE_MAP documentation and check for any additional convention docs:

7. user (2026-07-03T17:34:32.810Z)

```

1      <!--
2      PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3
4      A "tracked project doc" is how this repo carries substantial design/project work: it can
5      be
6      ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables,
7      then
8      archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
9      docs/GOLDEN_REGRESSION_FIXTURES.md.
10
11     Rules of thumb:
12     - §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable
13     PR.
14     - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs
15     sign-off),
16     or [design] (proposed). The doc reflects real shipped state, never aspirational.
17     - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
18     - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model
19     for
20     research- or security/privacy-heavy work). Delete sections that do not apply.
21     - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state
22     work).
23     -->
24
25     # <Project / Plan Title>
26
27     > **Status:** `DRAFT` ? `<owner-gated? | in progress>` . **Owner:** `<name>` .
28     **Created:** `<YYYY-MM-DD>` . **Last updated:** `<YYYY-MM-DD>`
29     > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
30     DONE)
31     > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
32     `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
33     > **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
34     > **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not
35     legal/financial advice where applicable.
36
37     ---
38
39     ## 0. TL;DR
40     <2?5 sentences: the goal, the approach, and the single most important caveat. Call out
41     any finding that is already actionable on its own.>
42
43     ## 1. Problem & goal
44     <Why now; the concrete user-facing outcome; what "good" looks like. Link the code/docs a

```

```

reader must read to get current.>
34
35    ## 2. Decisions locked
36    <Table ? capture answered questions so they are not relitigated.>
37
38    | # | Decision | Source / date | Implication |
39    |---|-----|-----|-----|
40    | D1 | ? | ? | ? |
41
42    ## 3. Foundation ? research / prior art *(OPTIONAL)*
43    <Include when the plan rests on external facts: cited research, legal/compliance,
benchmarks, prior-art survey. Mark researched-vs-verified; list any sign-off gates this
creates.>
44
45    ## 4. Design / architecture
46    <The actual plan. Data contracts, diagrams, and a REUSE MAP ? name the existing
modules/code this builds on so it is clear what is new vs reused.>
47
48    ## 5. Threat model / risk table *(OPTIONAL ? security / privacy / compliance work)*
49    <Who/what is protected vs NOT; residual risks stated plainly.>
50
51    ## 6. Honest limits ? what this does NOT do
52    <The false-confidence guardrail: what a green suite / a DONE status does NOT prove.>
53
54    ## 7. Deliverables & progress tracker  ? **source of truth**
55    Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
56
57    | ID | Deliverable | Depends on | Gated? | Status | PR |
58    |----|-----|-----|-----|-----|----|
59    | P0 | ? | ? | No | ? | ? |
60
61    ## 8. Open questions ? owner / external *(split blocking-gates from nice-to-knows; name
who decides)*
62    1. ?
63
64    ## 9. Definition of done
65    <Explicit acceptance checklist. When all ticked ? flip `Status ? DONE` and archive.>
66    - [ ] ?
67
68    ## 10. References
69    **Internal code anchors `[verified]`:** ``. **Internal docs:** ``.
**External:** ``.
70
71    ---
72
73    ### Changelog
74    - `` ? ``.
75

```

8. user (2026-07-03T17:34:33.269Z)

```

1    # Alpha Launch Readiness Plan
2
3    > **Status:** `DRAFT` - cross-repo execution tracker; **Owner:** @avidullu; **Created:**
2026-07-03; **Last updated:** 2026-07-03
4    > **Lifecycle:** `DRAFT` -> IN PROGRESS -> DONE -> archived`
5    > **Tracking anchors:** Section 7 is the progress tracker and source of truth; indexed
in `docs/README.md` and `docs/DOC_STATUS.md`.
6    > **Relation to existing docs:** consolidates the launch-relevant pending items from
`NEXT_STEPS.md`, `ROADMAP.md`, `docs/data_pipeline/GOLDEN_VIDEOS.md`,
`R2_EVAL_METRIC_DESIGN.md`, `COMPUTE_DECOUPLED_SERVING/`, and the sibling `kheIsutra` Studio
docs.
7    > **Honesty note:** `[verified]` means checked against the local synced repos on
2026-07-03. `[analysis]` means derived from a local command run on that snapshot. This is not

```

legal, financial, or launch approval.

8

9 ---

10

11 ## 0. TL;DR

12

13 The corpus is now large enough to move from "grow labels first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass. ``[verified]`` The golden manifest has 15 badminton videos; ``[analysis]`` the local manifest needed path rebasing before manifest-driven commands ran; ``[analysis]`` only 6 of 15 feature JSONs had the full fusion schema; ``[analysis]`` the n=15 quality baseline is stale and must be intentionally refreshed before it can gate anything.

14

15 The first controlled alpha should be a hosted, authenticated product loop: upload a video, process it, select rallies, compile, and download a reel. The owned-model/commercial rally-detection claim remains a separate NO-GO until the ship-gate target is ratified and the WASB checkpoint provenance is resolved.

16

17 ## 1. Problem & goal

18

19 The previous launch blockers were spread across roadmap docs and mixed together three different questions:

20

21 - ****Can an invited tester use the product without SSH?****

22 - ****Can the 15-video golden corpus now support honest quality gates?****

23 - ****Can we claim or ship an owned/commercial rally detector?****

24

25 This doc separates those into executable workstreams. The goal is to reach an invite-only alpha where the product is usable end-to-end, the quality claims are honest and reproducible, and the remaining model/legal gates are explicit rather than accidentally implied by launch activity.

26

27 ## 2. Decisions locked

28

29 | # | Decision | Source / date | Implication |

30 |---|---|---|---|

31 | D1 | ****Product alpha and owned-model launch are separate decisions.**** |

``NEXT_STEPS.md`` NO-GO banner, 2026-06-30; verified 2026-07-03 | We can launch a controlled hosted product loop without claiming the owned model is commercially cleared. |

32 | D2 | ****First alpha must be page-driven, not SSH/CLI-driven.**** | ``NEXT_STEPS.md`` P0 UI-driven/shareable item; verified 2026-07-03 | Hosted service, edge auth, upload, job polling, compile, and download are launch-critical. |

33 | D3 | ****The 15-video corpus is now a measurement asset, not a free launch pass.**** | ``[verified]`` ``docs/data_pipeline/GOLDEN_VIDEOS.md``; ``[analysis]`` local eval runs, 2026-07-03 | Rebase/portability, feature completeness, baseline refresh, and gate ratification come before quality claims. |

34 | D4 | ****Do not promote served Gate A by default from the current n=15 evidence.**** | ``[analysis]`` ``backend.eval.serve_contrast`` run, 2026-07-03 | Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever until owner ratification. |

35 | D5 | ****P10/P11 corpus promotion and train/re-eval are alpha-plus unless owner reopens D13.**** | sibling ``khehsutra/docs/STUDIO_MEDIA_LIFECYCLE.md``; verified 2026-07-03 | First alpha can use manual/offline corpus promotion; automated flywheel follows after serving and gates are stable. |

36

37 ## 3. Verified snapshot, 2026-07-03

38

39 All five Git checkouts were fast-forwarded before analysis:

40

41 | Repo | Branch | Synced HEAD |

42 |---|---|---|

43 | ``badminton-highlight-indexer`` | ``master`` | ``e4914d1`` |

44 | ``khehsutra`` | ``master`` | ``b0f70a5`` |

45 | ``rally-annotator`` | ``main`` | ``ef9e088`` |

46 | ``sports-data-collector`` | ``master`` | ``bfc014f`` |

47 | ``sports-obsreport`` | ``main`` | ``a0c7252`` |

```

48
49     Corpus/eval facts:
50
51     - `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says
52     `output/human_lovo_manifest.json` is the ingested golden corpus at `n = 15`.
53     - `[analysis]` The local manifest had stale absolute paths. A temporary rebased manifest
54     found all 15 trajectories and labels in this workspace.
55     - `[analysis]` There are 15 `output/*_golden_features.json` files, but only 6 carry full
56     shuttle/person/audio candidate features. The newer 9 are windowing-only schema and are skipped
57     by fusion/person eval.
58     - `[analysis]` `python -m backend.eval.fusion_compare --features-dir output` loaded only
59     6 videos and measured person-fusion `Delta F1 = -0.021`; not promotable.
60     - `[analysis]` `python -m backend.eval.serve_contrast --manifest <rebased-n15>` measured
61     proposal Gate A mean `Delta F1 +0.0287`, but served mean `Delta F1 -0.0328`; not served-safe.
62     - `[analysis]` `python -m training.gen0.harness --manifest <rebased-n15> --floor
63     eval_baselines/heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>` produced learned
64     LOVO mean F1 `0.551`, `ship=False`; blockers remain stale n=6 floor/sign test, uncalibrated ship
65     target, and WASB C3 provenance.
66     - `[analysis]` `python scripts/nightly_regression.py --manifest <rebased-n15>
67     --features-dir output --no-report` reported default/milestone tolF1 `0.3636` vs baseline-off
68     tolF1 `0.1382`, but marked the frozen baseline stale due to config/corpus changes.
69     - `[analysis]` `python -m backend.eval.ablation --features-dir output --granularity
70     group` currently fails at import due to a circular import between `backend/eval/ablation.py` and
71     `backend/eval/ablation_pdf.py`.
72
73     ## 4. Launch strategy
74
75     ### 4.1 Alpha lane: make the product usable
76
77     The alpha lane is successful when an invited tester can open the app, authenticate,
78     upload a video, watch the job progress, pick rallies, compile a reel, and download it. This lane
79     should avoid model-quality promises beyond the measured state and should default to the most
80     reliable serving path.
81
82     ### 4.2 Quality lane: make the 15-video corpus gate-ready
83
84     The quality lane is successful when the 15-video corpus is portable, reproducible,
85     feature-complete, and backed by refreshed baselines. Only then should R2 tolF1,
86     Gen-0/TemporalMaxer, or other model/promotion decisions be ratified.
87
88     ### 4.3 Flywheel lane: make tester feedback useful
89
90     The flywheel lane is successful when user corrections can flow into the collector/vault
91     path with consent/provenance and a visible re-eval delta. This is valuable for alpha learning,
92     but it should not block the first controlled alpha if manual promotion can bridge the gap.
93
94     ## 5. Risks and guardrails
95
96     | Risk | Why it matters | Guardrail |
97     |---|---|---|
98     | Launch activity implies model launch approval | The engine docs explicitly say rally-
99     detection product launch is NO-GO until ship target and WASB C3 clear | Keep all alpha copy
100    scoped to invite-only processing, not owned-model claims |
101    | Stale local manifests create false confidence | Manifest-driven tools skipped or
102    failed until paths were rebased | Make the corpus manifest portable and add a path/fixture smoke
103    check |
104    | 15-video corpus is only partially feature-complete | Fusion/person/audio eval is still
105    n=6 today | Regenerate or upconvert full-fusion features for all 15 before fusion decisions |
106    | Baseline refresh accidentally rubber-stamps a regression | Nightly runner correctly
107    reported stale baselines | Refresh baselines only after reviewer sign-off and record the
108    command/output |
109    | P10/P11 expands scope before hosted alpha is usable | Corpus flywheel is valuable but
110    can consume the alpha launch window | Keep P10/P11 behind explicit owner reopen unless alpha
111    lane is already stable |
112    | WASB checkpoint provenance remains unresolved | Blocks commercial sharing of a trained

```

model regardless of F1 | Treat C3 as a hard model-release gate; do not hide it inside product launch tasks |

84

85 ## 6. Honest limits

86

87 Completing this doc's alpha lane does ****not**** prove the owned model is ready, that WASB weights are commercially clean, that the product can serve unbounded public traffic, that multi-sport is ready, or that every tester correction automatically improves the model.

88

89 Completing the quality lane does ****not**** by itself deploy a service. It only makes the measurement surface honest enough for a launch decision.

90

91 ## 7. Deliverables & progress tracker

92

93 Legend: TODO = not started, IN PROGRESS = actively being worked, DONE = shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the row explicitly names a cross-repo batch.

94

ID	Deliverable	Repo / owner	Depends on	Gated?	Status	PR
A0	Create this alpha readiness tracker and index it in docs	`badminton-highlight-indexer`	-	No	DONE	#451
A1	Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth ON, upload -> process -> jobs -> compile -> download verified	`khesutra` + `badminton-highlight-indexer`	A0	Owner spend / CF config	TODO	-
A2	Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that fails loudly when labels or trajectories are missing	`badminton-highlight-indexer`	A0	No	IN PROGRESS	this PR
A3	Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile collector's six-video source manifest with the 15-video eval corpus	`sports-data-collector` + vault	A2	Owner upload/storage scope	TODO	-
A4	Refresh the n=15 nightly regression baseline intentionally and record the exact command/output; keep stale-baseline warnings until reviewed	`badminton-highlight-indexer`	A2	Reviewer sign-off	TODO	-
A5	Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for future Gen-0 comparisons	`badminton-highlight-indexer`	A2	No	TODO	-
A6	Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation on the n=15 corpus	`badminton-highlight-indexer`	A2	No	TODO	-
A7	Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so fusion/person/audio analyses run over all 15	`badminton-highlight-indexer`	A2	GPU/data availability	TODO	-
A8	Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly promote/reject person fusion, Gate A served-default, and any other R-series default flips	`badminton-highlight-indexer`	A6, A7	Reviewer sign-off	TODO	-
A9	Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce a non-serving artifact bundle and gate verdict	`badminton-highlight-indexer`	A5, A8	Owner approval for M0.4 scope	TODO	-
A10	Decide and record the alpha ship-gate target: metric, corpus slice, minimum threshold, significance rule, and what claims are allowed	Owner + `badminton-highlight-indexer`	A4, A5, A8	Owner decision	GATED	-
A11	Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred, document "no commercial model sharing" copy guardrail	Owner/legal	A10	Legal/provenance	GATED	-
A12	Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector promotion, `train`/`reeval` job, delta-F1 surface, and alignment with `COMPUTE_DECOUPLED_SERVING`	M11 `khesutra` + `badminton-highlight-indexer` + collector	A1, A3	Owner D13 reopen (`khesutra/docs/STUDIO_MEDIA_LIFECYCLE.md`)	GATED	-
A13	Alpha launch review: runbook, privacy/consent copy, observability/job-failure triage, known limits, rollback/disable steps, and owner GO/NO-GO	Cross-repo	A1, A4, A10, A11	Owner launch decision	TODO	-

111

112 ## 8. Concrete next PR sequence

113

- 114 1. ****Docs PR:**** land this tracker.
- 115 2. ****Corpus portability PR:**** rebase/portable manifest tooling plus missing-file smoke

```

test.
116     3. **Hosted alpha PR/batch:** health/auth/upload/process/job/compile/download
verification and runbook updates.
117     4. **Baseline PR:** intentional n=15 nightly baseline refresh and heuristic floor
refresh.
118     5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
119     6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy
limits, and launch checklist.
120
121     A2 smoke command:
122
123     ```powershell
124     python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15
125     python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15 --write output/human_lovo_manifest.local.json
126     ```
127
128     The checker resolves stale absolute paths against the repo's sibling `Annotation Setup`
folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is
absent.
129
130     ## 9. Open questions
131
132     1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or
should P10/P11 be reopened before any tester wave?
133     2. **Owner/platform:** Which hosted compute target is the next stand-up: Cloud Run GPU,
GCE L4 VM, or another approved GPU host?
134     3. **Owner/legal:** What is the acceptable alpha wording while WASB C3 remains
unresolved?
135     4. **Engineering:** Should the portable corpus manifest live as a generated artifact, a
checked-in redacted manifest, or both?
136     5. **Engineering:** Should n=15 feature regeneration target full fusion schema for every
clip, or should the eval loaders support mixed schemas with explicit capability tags?
137
138     ## 10. Definition of done
139
140     - [ ] Invited tester can complete the hosted record/upload -> reel download CUJ without
SSH.
141     - [ ] Alpha service is gated behind the chosen auth posture and has a documented
disable/rollback path.
142     - [ ] n=15 corpus is portable enough that manifest-driven quality commands do not rely
on stale local absolute paths.
143     - [ ] n=15 nightly baseline and heuristic floor are refreshed with recorded commands.
144     - [ ] R2/tolF1 gate is promoted or rejected with a recorded ablation result.
145     - [ ] Fusion/person/Gate-A served-default decisions are explicitly promoted or rejected
on the 15-video corpus.
146     - [ ] Ship-gate target and allowed alpha claims are recorded.
147     - [ ] WASB C3 status is recorded and reflected in alpha messaging.
148     - [ ] P10/P11 are either reopened with a committed sequence or explicitly deferred for
alpha-plus.
149     - [ ] Owner records GO/NO-GO for the first alpha wave.
150
151     ## 11. References
152
153     Internal docs:
154
155     - `docs/NEXT_STEPS.md`
156     - `docs/ROADMAP.md`
157     - `docs/data_pipeline/GOLDEN_VIDEOS.md`
158     - `docs/R2_EVAL_METRIC_DESIGN.md`
159     - `docs/RALLY_DETECTION_FIXES_PLAN.md`
160     - `docs/COMPUTE_DECOUPLED_SERVING/README.md`
161     - sibling `kheIsutra/NEXT_STEPS.md`

```



```

162 - sibling `khelsutra/docs/STUDIO_MEDIA_LIFECYCLE.md`
163 - sibling `khelsutra/docs/PER_RALLY_SELECTION.md`
164 - sibling `sports-data-collector/data/golden_source_videos.json`
165
166 Local verification commands used for the 2026-07-03 snapshot:
167
168 ```powershell
169 python -m backend.eval.fusion_compare --features-dir output
170 python -m backend.eval.serve_contrast --manifest <rebased-n15-manifest>
171 python -m training.gen0.harness --manifest <rebased-n15-manifest> --floor
eval_baselines\heuristic_lovo_n6.json --version analysis-n15 --out <temp-out>
172 python scripts\nightly_regression.py --manifest <rebased-n15-manifest> --features-dir
output --no-report
173 python -m backend.eval.ablation --features-dir output --granularity group
174 ```
175
176 ---
177
178 ### Changelog
179
180 - 2026-07-03 - Created from a synced five-repo review of `NEXT_STEPS`, `ROADMAP`,
golden-corpus docs, local eval artifacts, and the current 15-video corpus state.
181

```

9. user (2026-07-03T17:34:34.470Z)

```

backend/eval/calibrate_local.py
backend/eval/distill_local.py
backend/eval/fusion_golden.py
backend/eval/golden_manifest.py
backend/eval/rally_seg_eval.py
backend/eval/rally_seq_proto.py
backend/eval/serve_contrast.py
backend/eval/served_gate_a.py
docs/ALPHA_LAUNCH_READINESS.md
docs/data_pipeline/GOLDEN_VIDEOS.md
tests/test_fusion_compare.py
tests/test_fusion_golden_telemetry.py
tests/test_golden_manifest.py
tests/test_served_gate_a_regression.py
training/gen0/harness.py

```

10. assistant (2026-07-03T17:34:36.571Z)

Now let me examine the actual diff to check for convention violations:

11. user (2026-07-03T17:34:38.175Z)

```

backend/eval/calibrate_local.py      | 5 +-
backend/eval/distill_local.py        | 5 +-
backend/eval/fusion_golden.py         | 4 +-
backend/eval/golden_manifest.py       | 378 +++++
backend/eval/rally_seg_eval.py        | 4 +-
backend/eval/rally_seq_proto.py       | 5 +-
backend/eval/serve_contrast.py        | 4 +-
backend/eval/served_gate_a.py         | 5 +-
docs/ALPHA_LAUNCH_READINESS.md       | 11 +-
docs/data_pipeline/GOLDEN_VIDEOS.md  | 8 +-
tests/test_fusion_compare.py          | 4 +
tests/test_fusion_golden_telemetry.py | 3 +
tests/test_golden_manifest.py         | 158 +++++
tests/test_served_gate_a_regression.py | 21 +-
training/gen0/harness.py              | 4 +-
15 files changed, 586 insertions(+), 33 deletions(-)

```

12. assistant (2026-07-03T17:34:40.152Z)

Let me read the new golden_manifest.py file to check conventions:

13. user (2026-07-03T17:34:40.417Z)

```
1      """Portable golden-corpus manifest loader and smoke checker.
2
3      The owner-side golden manifest is intentionally gitignored and has historically
4      carried absolute paths from whichever box ingested a clip. This module keeps that
5      artifact usable after the workspace moves by resolving stale paths against the
6      current repo layout and failing loudly when required corpus files are absent.
7
8      Default local layout supported:
9      - ../Annotation Setup/Trajectories/<trajectory>.csv
10     - ../Annotation Setup/Golden Labelled/<labels>.csv
11     - output/<labels>.csv
12
13     Extra roots can be supplied with ``--root`` or ``RALLY_GOLDEN_SEARCH_ROOTS``.
14     """
15
16     from __future__ import annotations
17
18     import argparse
19     import json
20     import os
21     import sys
22     from dataclasses import dataclass
23     from pathlib import Path, PureWindowsPath
24     from typing import Any, Iterable, Sequence
25
26     DEFAULT_MANIFEST = "output/human_lovo_manifest.json"
27     SEARCH_ROOTS_ENV = "RALLY_GOLDEN_SEARCH_ROOTS"
28     DEFAULT_REQUIRED_FIELDS = ("trajectory", "labels")
29
30     _FIELD_DIR = {
31         "trajectory": "Trajectories",
32         "labels": "Golden Labelled",
33     }
34
35
36     @dataclass(frozen=True)
37     class ManifestIssue:
38         """One manifest path problem found by the resolver."""
39
40         name: str
41         field: str
42         path: str
43         reason: str
44
45         def message(self) -> str:
46             shown = f" {self.path!r}" if self.path else ""
47             return f"{self.name}: {self.field}{shown} - {self.reason}"
48
49
50     class ManifestPathError(RuntimeError):
51         """Raised when a strict manifest load finds missing required files."""
52
53         def __init__(self, manifest_path: str | os.PathLike[str], issues:
54             Sequence[ManifestIssue]):
55             self.manifest_path = os.fspath(manifest_path)
56             self.issues = list(issues)
57             detail = "\n".join(f" - {i.message()}" for i in self.issues[:12])
58             more = "" if len(self.issues) <= 12 else f"\n ... {len(self.issues) - 12} more"
59             super().__init__(
```

```

59         f"golden manifest path check failed for {self.manifest_path}: "
60         f"{len(self.issues)} issue(s)\n{n{detail}{more}}"
61     )
62
63
64 def repo_root() -> Path:
65     """Return the repository root for this module."""
66
67     return Path(__file__).resolve().parents[2]
68
69
70 def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
71     """Load the flat golden manifest list and validate its outer shape."""
72
73     path = Path(manifest_path)
74     with path.open(encoding="utf-8") as f:
75         data = json.load(f)
76     if not isinstance(data, list):
77         raise ValueError(f"golden manifest must be a JSON list: {path}")
78     out: list[dict[str, Any]] = []
79     for i, row in enumerate(data):
80         if not isinstance(row, dict):
81             raise ValueError(f"golden manifest row {i} is not an object: {path}")
82         out.append(dict(row))
83     return out
84
85
86 def resolve_manifest_paths(
87     manifest_path: str | os.PathLike[str],
88     *,
89     repo_root_path: str | os.PathLike[str] | None = None,
90     extra_roots: Iterable[str | os.PathLike[str]] = (),
91     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
92 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
93     """Load and rebase required path fields in a golden manifest.
94
95     Returns ``(resolved_specs, issues)``. The returned specs preserve all manifest
96     metadata but replace any resolved ``trajectory`` / ``labels`` path with the
97     existing local path. Missing required fields are reported in ``issues``.
98     """
99
100     manifest = load_manifest(manifest_path)
101     return resolve_manifest_specs(
102         manifest,
103         manifest_path,
104         repo_root_path=repo_root_path,
105         extra_roots=extra_roots,
106         required_fields=required_fields,
107     )
108
109
110 def load_resolved_manifest(
111     manifest_path: str | os.PathLike[str],
112     *,
113     repo_root_path: str | os.PathLike[str] | None = None,
114     extra_roots: Iterable[str | os.PathLike[str]] = (),
115     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
116     strict: bool = True,
117 ) -> list[dict[str, Any]]:
118     """Load a manifest with stale paths rebased; optionally raise on missing files."""
119
120     specs, issues = resolve_manifest_paths(
121         manifest_path,
122         repo_root_path=repo_root_path,
123         extra_roots=extra_roots,

```

```

124         required_fields=required_fields,
125     )
126     if strict and issues:
127         raise ManifestPathError(manifest_path, issues)
128     return specs
129
130
131 def resolve_manifest_specs(
132     specs: Sequence[dict[str, Any]],
133     manifest_path: str | os.PathLike[str],
134     *,
135     repo_root_path: str | os.PathLike[str] | None = None,
136     extra_roots: Iterable[str | os.PathLike[str]] = (),
137     required_fields: Sequence[str] = DEFAULT_REQUIRED_FIELDS,
138 ) -> tuple[list[dict[str, Any]], list[ManifestIssue]]:
139     """Resolve an already-loaded manifest list."""
140
141     manifest = Path(manifest_path)
142     root = Path(repo_root_path).resolve() if repo_root_path else repo_root()
143     roots = [Path(p) for p in _env_roots()]
144     roots.extend(Path(p) for p in extra_roots)
145     required = tuple(required_fields)
146
147     resolved: list[dict[str, Any]] = []
148     issues: list[ManifestIssue] = []
149     for idx, spec in enumerate(specs):
150         row = dict(spec)
151         name = str(row.get("name") or f"row[{idx}]")
152         for field in required:
153             raw = row.get(field)
154             if not raw:
155                 issues.append(ManifestIssue(name, field, "", "required field missing"))
156                 continue
157             hit = resolve_existing_path(
158                 str(raw),
159                 field,
160                 manifest_path=manifest,
161                 repo_root_path=root,
162                 extra_roots=roots,
163             )
164             if hit is None:
165                 issues.append(ManifestIssue(name, field, str(raw), "file not found"))
166             else:
167                 row[field] = str(hit)
168         resolved.append(row)
169     return resolved, issues
170
171
172 def resolve_existing_path(
173     raw_path: str,
174     field: str,
175     *,
176     manifest_path: str | os.PathLike[str],
177     repo_root_path: str | os.PathLike[str],
178     extra_roots: Iterable[str | os.PathLike[str]] = (),
179 ) -> Path | None:
180     """Resolve one manifest path field to an existing file, if possible."""
181
182     for candidate in _candidate_paths(
183         raw_path,
184         field,
185         Path(manifest_path),
186         Path(repo_root_path),
187         [Path(p) for p in extra_roots],
188     ):

```

```

189         if candidate.is_file():
190             return candidate.resolve()
191     return None
192
193
194 def _env_roots() -> list[Path]:
195     raw = os.environ.get(SEARCH_ROOTS_ENV, "")
196     if not raw:
197         return []
198     return [Path(p) for p in raw.split(os.pathsep) if p.strip()]
199
200
201 def _basename(raw_path: str) -> str:
202     # PureWindowsPath handles stale Windows paths even when tests run on POSIX.
203     win_name = PureWindowsPath(raw_path).name
204     return win_name or Path(raw_path).name
205
206
207 def _candidate_paths(
208     raw_path: str,
209     field: str,
210     manifest_path: Path,
211     root: Path,
212     extra_roots: Sequence[Path],
213 ) -> list[Path]:
214     basename = _basename(raw_path)
215     raw = Path(os.path.expandvars(os.path.expanduser(raw_path)))
216     manifest_dir = manifest_path.resolve().parent
217     candidates: list[Path] = [raw]
218
219     if not raw.is_absolute():
220         candidates.extend([manifest_dir / raw, root / raw])
221
222     for directory in _candidate_dirs(field, manifest_dir, root, extra_roots):
223         candidates.append(directory / basename)
224     return _dedup_paths(candidates)
225
226
227 def _candidate_dirs(
228     field: str, manifest_dir: Path, root: Path, extra_roots: Sequence[Path]
229 ) -> list[Path]:
230     field_dir = _FIELD_DIR.get(field)
231     dirs: list[Path] = [manifest_dir]
232     if field_dir:
233         dirs.extend(
234             [
235                 root.parent / "Annotation Setup" / field_dir,
236                 root.parent / "Annotation Setup" / "Collect" / field_dir,
237                 root / "output",
238             ]
239         )
240     dirs.extend(root_dirs for r in extra_roots for root_dirs in _expand_root(r, field))
241     return _dedup_paths(dirs)
242
243
244 def _expand_root(root: Path, field: str) -> list[Path]:
245     field_dir = _FIELD_DIR.get(field)
246     dirs = [root, root / "output"]
247     if field_dir:
248         dirs.extend(
249             [
250                 root / field_dir,
251                 root / "Annotation Setup" / field_dir,
252                 root / "Annotation Setup" / "Collect" / field_dir,
253             ]

```

```

254         )
255     return dirs
256
257
258 def _dedup_paths(paths: Iterable[Path]) -> list[Path]:
259     out: list[Path] = []
260     seen: set[str] = set()
261     for path in paths:
262         try:
263             key = os.path.normcase(os.path.abspath(os.fspath(path)))
264         except OSError:
265             key = os.fspath(path)
266         if key in seen:
267             continue
268         seen.add(key)
269         out.append(path)
270     return out
271
272
273 def write_manifest(path: str | os.PathLike[str], specs: Sequence[dict[str, Any]]) ->
None:
274     """Write a resolved manifest JSON atomically enough for local tooling."""
275
276     out = Path(path)
277     out.parent.mkdir(parents=True, exist_ok=True)
278     tmp = out.with_name(out.name + ".tmp")
279     with tmp.open("w", encoding="utf-8") as f:
280         json.dump(list(specs), f, indent=2)
281         f.write("\n")
282     os.replace(tmp, out)
283
284
285 def build_parser() -> argparse.ArgumentParser:
286     p = argparse.ArgumentParser(
287         description="Check/rebase the local golden corpus manifest paths."
288     )
289     p.add_argument("--manifest", default=DEFAULT_MANIFEST)
290     p.add_argument(
291         "--repo-root",
292         default=None,
293         help="repo root used for default sibling output/Annotation Setup lookups",
294     )
295     p.add_argument(
296         "--root",
297         action="append",
298         default=[],
299         help=(
300             "additional search root; repeatable. Also accepts roots via "
301             f"{SEARCH_ROOTS_ENV} separated by os.pathsep."
302         ),
303     )
304     p.add_argument(
305         "--require",
306         action="append",
307         choices=sorted(_FIELD_DIR),
308         default=None,
309         help="path field to require; repeatable. Defaults to trajectory and labels.",
310     )
311     p.add_argument(
312         "--min-count",
313         type=int,
314         default=None,
315         help="fail unless at least this many manifest rows are present",
316     )
317     p.add_argument(

```

```

318         "--write",
319         default=None,
320         help="optional path to write the resolved local manifest JSON",
321     )
322     return p
323
324
325 def main(argv: Sequence[str] | None = None) -> int:
326     args = build_parser().parse_args(argv)
327     required = tuple(args.require or DEFAULT_REQUIRED_FIELDS)
328     try:
329         original = load_manifest(args.manifest)
330         resolved, issues = resolve_manifest_specs(
331             original,
332             args.manifest,
333             repo_root_path=args.repo_root,
334             extra_roots=args.root,
335             required_fields=required,
336         )
337     except (OSError, ValueError, json.JSONDecodeError) as e:
338         print(f"ERROR: {e}", file=sys.stderr)
339         return 2
340
341     if args.min_count is not None and len(resolved) < args.min_count:
342         issues.append(
343             ManifestIssue(
344                 "manifest",
345                 "rows",
346                 str(len(resolved)),
347                 f"expected at least {args.min_count} row(s)",
348             )
349         )
350
351     if issues:
352         print(
353             f"ERROR: golden manifest path check failed ({len(issues)} issue(s)):",
354             file=sys.stderr,
355         )
356         for issue in issues:
357             print(f" - {issue.message()}", file=sys.stderr)
358         return 2
359
360     rebased = sum(
361         1
362         for before, after in zip(original, resolved)
363         for field in required
364         if str(before.get(field, "")) != str(after.get(field, ""))
365     )
366     print(
367         f"OK: {args.manifest} has {len(resolved)} video(s); "
368         f"{len(resolved) * len(required)} required path(s) present; "
369         f"{rebased} path(s) rebased."
370     )
371     if args.write:
372         write_manifest(args.write, resolved)
373         print(f"wrote resolved manifest: {args.write}")
374     return 0
375
376
377 if __name__ == "__main__":
378     raise SystemExit(main())
379

```

```

1     import json
2     from pathlib import Path
3
4     import pytest
5
6     from backend.eval import golden_manifest as gm
7
8
9     def _write(path: Path, body: str = "x\n") -> Path:
10         path.parent.mkdir(parents=True, exist_ok=True)
11         path.write_text(body, encoding="utf-8")
12         return path
13
14
15     def _write_manifest(repo: Path, rows: list[dict]) -> Path:
16         path = repo / "output" / "human_lovo_manifest.json"
17         path.parent.mkdir(parents=True, exist_ok=True)
18         path.write_text(json.dumps(rows), encoding="utf-8")
19         return path
20
21
22     def test_resolves_stale_absolute_paths_to_workspace_layout(tmp_path):
23         workspace = tmp_path / "workspace"
24         repo = workspace / "badminton-highlight-indexer"
25         repo.mkdir(parents=True)
26         traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo_traj.csv")
27         old_label = _write(
28             workspace / "Annotation Setup" / "Golden Labelled" / "Foo.rallies.csv"
29         )
30         output_label = _write(repo / "output" / "Bar.rallies.csv")
31         manifest = _write_manifest(
32             repo,
33             [
34                 {
35                     "name": "foo",
36                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
37                     "labels": "C:/old/Annotation Setup/Collect/Golden
Labelled/Foo.rallies.csv",
38                     "fps": 30,
39                     "frame_width": 1920,
40                 },
41                 {
42                     "name": "bar",
43                     "trajectory": "C:/old/Annotation
Setup/Collect/Trajectories/foo_traj.csv",
44                     "labels": "C:/old/badminton-highlight-indexer/output/Bar.rallies.csv",
45                     "fps": 30,
46                     "frame_width": 1920,
47                 },
48             ],
49         )
50
51         specs, issues = gm.resolve_manifest_paths(manifest, repo_root_path=repo)
52
53         assert issues == []
54         assert Path(specs[0]["trajectory"]) == traj.resolve()
55         assert Path(specs[0]["labels"]) == old_label.resolve()
56         assert Path(specs[1]["labels"]) == output_label.resolve()
57
58
59     def test_strict_loader_reports_missing_required_files(tmp_path):
60         repo = tmp_path / "repo"
61         repo.mkdir()
62         manifest = _write_manifest(

```



```

63         repo,
64         [
65             {
66                 "name": "missing",
67                 "trajectory": "C:/old/missing_traj.csv",
68                 "labels": "C:/old/missing.rallies.csv",
69             }
70         ],
71     )
72
73     with pytest.raises(gm.ManifestPathError) as excinfo:
74         gm.load_resolved_manifest(manifest, repo_root_path=repo)
75
76     assert len(excinfo.value.issues) == 2
77     assert "file not found" in str(excinfo.value)
78
79
80 def test_trajectory_only_resolution_leaves_label_optional(tmp_path):
81     workspace = tmp_path / "workspace"
82     repo = workspace / "badminton-highlight-indexer"
83     repo.mkdir(parents=True)
84     traj = _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
85     manifest = _write_manifest(
86         repo,
87         [
88             {
89                 "name": "foo",
90                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
91                 "labels": "C:/old/missing.rallies.csv",
92             }
93         ],
94     )
95
96     specs, issues = gm.resolve_manifest_paths(
97         manifest, repo_root_path=repo, required_fields=("trajectory",)
98     )
99
100     assert issues == []
101     assert Path(specs[0]["trajectory"]) == traj.resolve()
102     assert specs[0]["labels"] == "C:/old/missing.rallies.csv"
103
104
105 def test_cli_writes_resolved_manifest(tmp_path, capsys):
106     workspace = tmp_path / "workspace"
107     repo = workspace / "badminton-highlight-indexer"
108     repo.mkdir(parents=True)
109     _write(workspace / "Annotation Setup" / "Trajectories" / "foo.csv")
110     _write(repo / "output" / "Foo.rallies.csv")
111     manifest = _write_manifest(
112         repo,
113         [
114             {
115                 "name": "foo",
116                 "trajectory": "C:/old/Annotation Setup/Collect/Trajectories/foo.csv",
117                 "labels": "C:/old/badminton-highlight-indexer/output/Foo.rallies.csv",
118             }
119         ],
120     )
121     out = repo / "output" / "human_lovo_manifest.local.json"
122
123     rc = gm.main(
124         [
125             "--manifest",
126             str(manifest),
127             "--repo-root",

```

```

128         str(repo),
129         "--min-count",
130         "1",
131         "--write",
132         str(out),
133     ]
134 )
135
136 captured = capsys.readouterr()
137 assert rc == 0
138 assert "OK:" in captured.out
139 assert out.is_file()
140 resolved = json.loads(out.read_text(encoding="utf-8"))
141 assert Path(resolved[0]["trajectory"]).is_file()
142 assert Path(resolved[0]["labels"]).is_file()
143
144
145 def test_cli_fails_loudly_on_missing_file(tmp_path, capsys):
146     repo = tmp_path / "repo"
147     repo.mkdir()
148     manifest = _write_manifest(
149         repo,
150         [{"name": "foo", "trajectory": "missing.csv", "labels": "missing.rallies.csv"}],
151     )
152
153     rc = gm.main(["--manifest", str(manifest), "--repo-root", str(repo)])
154
155     captured = capsys.readouterr()
156     assert rc == 2
157     assert "golden manifest path check failed" in captured.err
158     assert "foo: trajectory" in captured.err
159

```

15. assistant (2026-07-03T17:34:42.698Z)

Now let me check the modified backend/eval files and other changed files for convention violations:

16. user (2026-07-03T17:34:44.633Z)

```

diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
     from __future__ import annotations

     import argparse
-    import json
     from typing import Callable, Dict, List, Tuple

     from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
 from backend.pipeline.detectors import rally_gate
 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:

```

```

-         specs = json.load(f)
+     specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, metric=args.metric)

diff --git a/backend/eval/distill_local.py b/backend/eval/distill_local.py
index 52ac85a..a05b3ff 100644
--- a/backend/eval/distill_local.py
+++ b/backend/eval/distill_local.py
@@ -24,13 +24,13 @@ Run:
    from __future__ import annotations

    import argparse
-import json
    from typing import Dict, List, Optional, Tuple

    from backend.eval import windowing as _windowing
    from backend.eval.calibrate_wasb import load_golden_gts
    from backend.eval.classifier import LogisticRegression
    from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.metrics import score
    from backend.eval.training import label_candidates
    from backend.pipeline.detectors import rally_gate
@@ -218,8 +218,7 @@ def main() -> None:
    )
    ap.add_argument("--model-out", default="output/local_rally_model.json")
    args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
    run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py
index dd3f6a1..aee6d7e 100644
--- a/backend/eval/fusion_golden.py
+++ b/backend/eval/fusion_golden.py
@@ -25,6 +25,7 @@ from typing import Any, Dict, List, Optional

    from backend.eval import distill_local
    from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.training import label_candidates
    from backend.pipeline.detectors import fusion_features as ff
    from backend.pipeline.detectors import rally_gate
@@ -160,8 +161,7 @@ def run(
    fresh: bool = False,
    smallest_first: bool = False,
) -> List[str]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).

```

17. assistant (2026-07-03T17:34:46.700Z)

Now let me check the docs changes and look for any doc status/honesty tags:

18. user (2026-07-03T17:34:48.643Z)

```
diff --git a/docs/ALPHA_LAUNCH_READINESS.md b/docs/ALPHA_LAUNCH_READINESS.md
```

```

index 04d46d2..bcc7fea 100644
--- a/docs/ALPHA_LAUNCH_READINESS.md
+++ b/docs/ALPHA_LAUNCH_READINESS.md
@@ -96,7 +96,7 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/
|----|-----|-----|-----|-----|----|
| A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-indexer`
| - | No | DONE | #451 |
| A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check, edge auth
ON, upload -> process -> jobs -> compile -> download verified | `khelsutra` + `badminton-
highlight-indexer` | A0 | Owner spend / CF config | TODO | - |
-| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
TODO | - |
+| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
IN PROGRESS | this PR |
| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile
collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` +
vault | A2 | Owner upload/storage scope | TODO | - |
| A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
| A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for
future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |
@@ -118,6 +118,15 @@ Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/
5. **Ablation/fusion PRs:** fix ablation import, complete 15-video features, rerun
promotion/rejection decisions.
6. **Launch decision PR/docs update:** record ship-gate target, C3 status, alpha copy limits,
and launch checklist.

+A2 smoke command:
+
+```powershell
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15 --write output/human_lovo_manifest.local.json
+```
+
+The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders
and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.
+
## 9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should
P10/P11 be reopened before any tester wave?
diff --git a/docs/data_pipeline/GOLDEN_VIDEOS.md b/docs/data_pipeline/GOLDEN_VIDEOS.md
index 6d68eeb..bb197bd 100644
--- a/docs/data_pipeline/GOLDEN_VIDEOS.md
+++ b/docs/data_pipeline/GOLDEN_VIDEOS.md
@@ -22,6 +22,11 @@ auditable. Updated whenever a new video is labeled + ingested.
- **Ingestion** = copy the trajectory + labels to durable paths, write
`output/<name>_golden_features.json`
(`gts` + candidates), append a row to `output/human_lovo_manifest.json`. (`scratch/at_par.py`
scores a
single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole corpus.)
+- **Path smoke check:** before running corpus evals, run
+ `python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15`.
+ The checker rebases stale absolute paths against sibling `Annotation Setup` folders and
`output/`, and
+ exits non-zero if any required trajectory or labels file is missing. Add `--write
output/human_lovo_manifest.local.json`

```

```
+ when a resolved local manifest artifact is useful.
- **Guardrails:** owned footage only; **minors excluded**; labels never derived from a paid LLM
(Gemini is
  inference/reference only). Commercial-clean.
```

```
@@ -57,4 +62,5 @@ auditable. Updated whenever a new video is labeled + ingested.
## How to isolate the golden videos
Each row's **source proxy** column is the file to copy into a dedicated golden-videos location.
The labels
  live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
-trajectory is in `Collect/Trajectories/` (path in the manifest). Keep this table in sync on
every new ingest.
+trajectory is in `Annotation Setup/Trajectories/` (older manifests may still say
`Collect/Trajectories/`;
+use the path smoke checker above). Keep this table in sync on every new ingest.
```

19. assistant (2026-07-03T17:34:50.519Z)

Let me check the test file changes:

20. user (2026-07-03T17:34:52.405Z)

```
diff --git a/tests/test_fusion_compare.py b/tests/test_fusion_compare.py
index d3477c8..f042eea 100644
--- a/tests/test_fusion_compare.py
+++ b/tests/test_fusion_compare.py
@@ -203,6 +203,8 @@ def test_fusion_golden_run_is_resumable(tmp_path, monkeypatch):

    out_dir = str(tmp_path / "out")
    os.makedirs(out_dir)
+   for name in ("a.csv", "b.csv", "a.rallies.csv", "b.rallies.csv"):
+       (tmp_path / name).write_text("x\n", encoding="utf-8")
    manifest = [
        {
            "name": "alpha",
@@ -267,6 +269,8 @@ def test_fusion_golden_smallest_first(tmp_path, monkeypatch):
    os.makedirs(out_dir)
    (tmp_path / "big.csv").write_text("x" * 2000)
    (tmp_path / "small.csv").write_text("x" * 20)
+   (tmp_path / "b.rallies.csv").write_text("x\n", encoding="utf-8")
+   (tmp_path / "s.rallies.csv").write_text("x\n", encoding="utf-8")
    manifest = [
        {
            "name": "big",
diff --git a/tests/test_fusion_golden_telemetry.py b/tests/test_fusion_golden_telemetry.py
index 1fab90..bdefc88 100644
--- a/tests/test_fusion_golden_telemetry.py
+++ b/tests/test_fusion_golden_telemetry.py
@@ -45,6 +45,9 @@ def _patch_telemetry(monkeypatch, gpu_fn=_fake_gpu, sys_fn=_fake_sys):

    def _write_manifest(tmp_path, names):
+       for n in names:
+           (tmp_path / f"{n}.csv").write_text("x\n", encoding="utf-8")
+           (tmp_path / f"{n}.rallies.csv").write_text("x\n", encoding="utf-8")
        manifest = [
            {
                "name": n,
diff --git a/tests/test_served_gate_a_regression.py b/tests/test_served_gate_a_regression.py
index 7fd7630..56848df 100644
--- a/tests/test_served_gate_a_regression.py
+++ b/tests/test_served_gate_a_regression.py
@@ -37,7 +37,6 @@ re-measurement as the corpus grows.
```

```

from __future__ import annotations

-import json
import os

import pytest
@@ -49,6 +48,10 @@ pytest.importorskip("cv2")
    from backend.eval import segmentation_metrics as _segm # noqa: E402
    from backend.eval import served_gate_a as sga # noqa: E402
    from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
+from backend.eval.golden_manifest import ( # noqa: E402
+    ManifestPathError,
+    load_resolved_manifest,
+)
    from backend.eval.metrics import score # noqa: E402
    from backend.pipeline.detectors import rally_gate # noqa: E402
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
@@ -64,18 +67,14 @@ MANIFEST = os.environ.get(

def _golden_specs():
-    """Return the manifest specs whose trajectory + label files are all present, else []."""
+    """Return rebased manifest specs when trajectory + label files are present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
-    with open(MANIFEST, encoding="utf-8") as f:
-        specs = json.load(f)
-    ok = [
-        s
-        for s in specs
-        if os.path.isfile(str(s.get("trajectory", "")))
-        and os.path.isfile(str(s.get("labels", "")))
-    ]
-    return ok if len(ok) >= 6 else []
+    try:
+        specs = load_resolved_manifest(MANIFEST)
+    except ManifestPathError:
+        return []
+    return specs if len(specs) >= 6 else []

_SPECS = _golden_specs()
diff --git a/training/gen0/harness.py b/training/gen0/harness.py
index 6eec29f..433d2cb 100644
--- a/training/gen0/harness.py
+++ b/training/gen0/harness.py
@@ -36,6 +36,7 @@ from typing import Dict, List, Optional
import numpy as np

    from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
    from backend.eval.gt_loader import load_gt_intervals
    from backend.eval.rally_seq_proto import (
        _seg_f1,
@@ -217,8 +218,7 @@ def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
    def run(
        manifest_path: str, out_dir: str, version: str, floor_path: Optional[str] = None
    ) -> dict:
-        with open(manifest_path, encoding="utf-8") as f:
-            specs = json.load(f)
+        specs = load_resolved_manifest(manifest_path)
        floor = None
        if floor_path:
            with open(floor_path, encoding="utf-8") as f:

```

21. assistant (2026-07-03T17:34:54.324Z)

Now let me check if there are any other backend/eval files and see if there are specific conventions about how new code should be tagged:

22. user (2026-07-03T17:34:55.226Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\eval'

23. user (2026-07-03T17:34:56.677Z)

""One-command rally-segmentation eval ? score a windowing against the golden labels with the over-segmentation-sensitive metric set, honestly.

This is the harness the rally-quality project (`docs/RALLY\_QUALITY\_RESEARCH.md` \$4.5 / \$7) measures every tier against. It closes the loop the over-merge finding exposed: plain temporal IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k, the segment-count ratio, and the explicit `merge\_split` breakdown (`segmentation\_metrics`).

For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds candidate windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the SERVED operating point and any future windowing the same way) ? scores the windows-as-rallies against the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI, and can diff two windowings with a paired sign-test + CI (the per-tier `delta`).

GROUND TRUTH: the golden manifest (`output/human\_lovo\_manifest.json`) gives each video's trajectory path + fps + frame_width; the GT rally intervals come from `output/<name>\_golden\_features.json` (the `gts` key) so this needs no extra label files.

Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
""

```
from __future__ import annotations
```

```
import argparse
import json
import os
from typing import Any, Dict, List, Optional, Tuple
```

```
from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import Interval
from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
```

```
DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
DEFAULT_FEATURES_DIR = "output"
DEFAULT_IOU = 0.5
```

```
#: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
#: (end ? start) exceeds this. The local detector over-fires, and its false positives are mostly
#: SHORT (motion blips, background-court fragments on multicourt footage); filtering to long
rallies
#: strips most of those FPs and reveals the real product quality on the long-rally happy path.
Used
#: as the default split point for the stratified report (`--long-tau` overrides it). DURATION,
not
#: shot-count, is the filter: golden `shots_count` is unlabeled and the no-AI path reports it
as
#: "Unknown", whereas duration is derived from the same start/end labels we already trust.
LONG_RALLY_TAU = 5.0
```

```
#: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
```

#: (``output/human\_lovo\_manifest.json``) is gitignored, so this committed set is the AUDITABLE source
#: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN\_VIDEOS.md`` (the clips tagged
#: "multicourt"). Multicourt footage carries background games on adjacent courts ? the dominant

24. assistant (2026-07-03T17:34:58.676Z)

Let me look at the full diffs to understand all changes and check for any documentation honesty-tag issues:

25. user (2026-07-03T17:35:00.459Z)

```
diff --git a/backend/eval/rally_seg_eval.py b/backend/eval/rally_seg_eval.py
index 0acca2d..3846ce0 100644
--- a/backend/eval/rally_seg_eval.py
+++ b/backend/eval/rally_seg_eval.py
@@ -27,6 +27,7 @@ import os
     from typing import Any, Dict, List, Optional, Tuple

     from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
+from backend.eval.golden_manifest import load_resolved_manifest
     from backend.eval.metrics import Interval
     from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

@@ -185,8 +186,7 @@ def load_golden(
 ) -> List[Dict[str, Any]]:
     """Load each golden video's trajectory path + fps + frame_width (manifest) and GT rally
     intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
     flagged."""
-    with open(manifest_path) as fh:
-        manifest = json.load(fh)
+    manifest = load_resolved_manifest(manifest_path, required_fields=("trajectory",))
     out: List[Dict[str, Any]] = []
     for v in manifest:
         feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
diff --git a/backend/eval/rally_seq_proto.py b/backend/eval/rally_seq_proto.py
index 2ea9483..b017f94 100644
--- a/backend/eval/rally_seq_proto.py
+++ b/backend/eval/rally_seq_proto.py
@@ -29,7 +29,6 @@ Run:
     from __future__ import annotations

     import argparse
-    import json
     from typing import Dict, List, Tuple

     import numpy as np
@@ -38,6 +37,7 @@ from scipy.stats import rankdata

     from backend.eval.calibrate_wasb import load_golden_gts
     from backend.eval.classifier import LogisticRegression
+from backend.eval.golden_manifest import load_resolved_manifest
     from backend.eval.metrics import score

     # Feature extraction is single-sourced in backend/features (ADR-008): training and
@@ -196,8 +196,7 @@ def main() -> None:
     help="JSON: [{name,trajectory,fps,frame_width,labels}, ...]",
 )
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        run(json.load(f))
+    run(load_resolved_manifest(args.manifest))
```



```

if __name__ == "__main__":
diff --git a/backend/eval/serve_contrast.py b/backend/eval/serve_contrast.py
index 328857b..0f1b1f4 100644
--- a/backend/eval/serve_contrast.py
+++ b/backend/eval/serve_contrast.py
@@ -32,6 +32,7 @@ from typing import Any, Dict, List, Sequence, Tuple
from backend.eval import windowing as W
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.gemini_refine import merge_overlaps
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import score
from backend.eval.promotion import promote_if_served_safe
from backend.pipeline.detectors import rally_gate
@@ -194,8 +195,7 @@ def main() -> None:
    )
    args = ap.parse_args()

-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
    result = run_contrast(
        specs,
        iou=args.iou,
diff --git a/backend/eval/served_gate_a.py b/backend/eval/served_gate_a.py
index 994b9c1..c2151fb 100644
--- a/backend/eval/served_gate_a.py
+++ b/backend/eval/served_gate_a.py
@@ -23,13 +23,13 @@ Run (owner box, golden trajectories present):
from __future__ import annotations

import argparse
-import json
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from backend.config.models import FusionConfig, IndexingConfig, WasbIndexConfig
from backend.eval import segmentation_metrics as _segm
from backend.eval.calibrate_wasb import load_golden_gts
+from backend.eval.golden_manifest import load_resolved_manifest
from backend.eval.metrics import Interval, Score, score
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
@@ -196,8 +196,7 @@ def format_table(
def run(
    manifest_path: str, iou: float = 0.5
) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
-    with open(manifest_path, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(manifest_path)
    results = evaluate_manifest(specs, iou=iou)
    return results, aggregate(results)

```

26. assistant (2026-07-03T17:35:02.696Z)

Now let me check the README.md to see if there's a docs indexing requirement:

27. user (2026-07-03T17:35:04.556Z)

```

18:- [ALPHA_LAUNCH_READINESS.md](ALPHA_LAUNCH_READINESS.md) ? *** Active tracked project
(`DRAFT`, created 2026-07-03):** cross-repo alpha readiness tracker after the golden corpus
reached 15 videos. Separates the hosted product-alpha lane (shareable upload?process?pick?reel)
from the owned-model/commercial rally-detection gate (ship target + WASB-C3 still unresolved).
$7 is the source of truth for the concrete next PR sequence: corpus portability, n=15 baseline
refresh, ablation/fusion reruns, hosted serving, alpha copy/launch review.

```

28. assistant (2026-07-03T17:35:08.293Z)

Good. Now let me check if there are any doc status or honesty tags referenced in CLAUDE.md or PROJECT_DOC_TEMPLATE:

29. user (2026-07-03T17:35:09.606Z)

```
docs/PROJECT_DOC_TEMPLATE.md:11:- Keep it HONEST: tag each claim [verified] (checked against
code), [researched] (needs sign-off),
docs/PROJECT_DOC_TEMPLATE.md:12: or [design] (proposed). The doc reflects real shipped state,
never aspirational.
docs/PROJECT_DOC_TEMPLATE.md:25:> **Honesty note:** mark each claim `[verified]` /
`[researched]` / `[design]`. Not legal/financial advice where applicable.
docs/PROJECT_DOC_TEMPLATE.md:69:**Internal code anchors `[verified]`:** `` . **Internal
docs:** `` . **External:** `` .
docs/ALPHA_LAUNCH_READINESS.md:7:> **Honesty note:** `[verified]` means checked against the
local synced repos on 2026-07-03. `[analysis]` means derived from a local command run on that
snapshot. This is not legal, financial, or launch approval.
docs/ALPHA_LAUNCH_READINESS.md:13:The corpus is now large enough to move from "grow labels
first" into alpha-readiness work, but the 15-video headline is not itself a launch gate pass.
`[verified]` The golden manifest has 15 badminton videos; `[analysis]` the local manifest needed
path rebasing before manifest-driven commands ran; `[analysis]` only 6 of 15 feature JSONs had
the full fusion schema; `[analysis]` the n=15 quality baseline is stale and must be
intentionally refreshed before it can gate anything.
docs/ALPHA_LAUNCH_READINESS.md:33:| D3 | **The 15-video corpus is now a measurement asset, not a
free launch pass.** | `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md`; `[analysis]` local
eval runs, 2026-07-03 | Rebase/portability, feature completeness, baseline refresh, and gate
ratification come before quality claims. |
docs/ALPHA_LAUNCH_READINESS.md:34:| D4 | **Do not promote served Gate A by default from the
current n=15 evidence.** | `[analysis]` `backend.eval.serve_contrast` run, 2026-07-03 |
Proposal-side lift does not survive served-windowing safety; keep it as an opt-in/analysis lever
until owner ratification. |
docs/ALPHA_LAUNCH_READINESS.md:51:- `[verified]` `docs/data_pipeline/GOLDEN_VIDEOS.md` says
`output/human_lovo_manifest.json` is the ingested golden corpus at `n = 15`.
docs/ALPHA_LAUNCH_READINESS.md:52:- `[analysis]` The local manifest had stale absolute paths. A
temporary rebased manifest found all 15 trajectories and labels in this workspace.
docs/ALPHA_LAUNCH_READINESS.md:53:- `[analysis]` There are 15 `output/*_golden_features.json`
files, but only 6 carry full shuttle/person/audio candidate features. The newer 9 are windowing-
only schema and are skipped by fusion/person eval.
docs/ALPHA_LAUNCH_READINESS.md:54:- `[analysis]` `python -m backend.eval.fusion_compare
--features-dir output` loaded only 6 videos and measured person-fusion `Delta F1 = -0.021`; not
promotable.
docs/ALPHA_LAUNCH_READINESS.md:55:- `[analysis]` `python -m backend.eval.serve_contrast
--manifest <rebased-n15>` measured proposal Gate A mean `Delta F1 +0.0287`, but served mean
`Delta F1 -0.0328`; not served-safe.
docs/ALPHA_LAUNCH_READINESS.md:56:- `[analysis]` `python -m training.gen0.harness --manifest
<rebased-n15> --floor eval_baselines/heuristic_lovo_n6.json --version analysis-n15 --out <temp-
out>` produced learned LOVO mean F1 `0.551`, `ship=False`; blockers remain stale n=6 floor/sign
test, uncalibrated ship target, and WASB C3 provenance.
docs/ALPHA_LAUNCH_READINESS.md:57:- `[analysis]` `python scripts/nightly_regression.py
--manifest <rebased-n15> --features-dir output --no-report` reported default/milestone tolF1
`0.3636` vs baseline-off tolF1 `0.1382`, but marked the frozen baseline stale due to
config/corpus changes.
docs/ALPHA_LAUNCH_READINESS.md:58:- `[analysis]` `python -m backend.eval.ablation --features-dir
output --granularity group` currently fails at import due to a circular import between
`backend/eval/ablation.py` and `backend/eval/ablation_pdf.py`.
```

30. assistant (2026-07-03T17:35:12.743Z)

Now let me check the CLAUDE.md for backend/eval conventions about placement:

31. user (2026-07-03T17:35:13.933Z)

20 ## 0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

```

21
22     Top-level tree and the placement rule for each kind of file. When unsure, prefer an
existing
23     registry / extension point (`?`) over a new top-level file.
24
25     | You're adding? | Put it in? | Notes |
26     |---|---|---|
27     | Backend / runtime code | `backend/<subsystem>/` |
`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`,
`infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most
additions are register a class in the relevant `? registry, not a new module tree. |
28     | Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature) |
`backend/eval/` | Pure core, no side-effects at import; CLI/I-O lives in a separate driver.
These are imported widely (e.g. `calibrate_wasb` by 8 sites) ? they are library, not
scripts; do NOT move them out. |
29     | Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app
code) | scripts/` | e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`.
Run as `python scripts/<name>.py`. If it needs `load_dotenv()/`sys.path` before importing
backend, add `scripts/<name>.py" = ["E402"]` to `ruff.toml`. |
30     | Ops / maintenance tool (not part of the served app ? cache GC, batch admin) |
`tools/` | Run as `python -m tools.<name>` (it's an importable package). e.g.
`tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested. |
31     | Training code (model fitting) | `training/` | Behind the CI-enforced import wall
(ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`. |
32     | A test | `tests/test_*.py` | pytest auto-discovers; the full-suite lane has a 70%
coverage floor. Pure/offline + mocked (no GPU/WSL/network). |
33     | A doc (design, plan, living log) | `docs/` | Index it in `docs/README.md`. Only
terminal-state (DONE/REJECTED) work moves to `docs/archives/`. |
34     | A throwaway repro / debug script | `scratch/` | Gitignored + excluded from
ruff/pytest. Never imported by app code. |
35     | A model artifact (weights + manifest) | `artifacts/<name>/<ver>/` | ADR-008:
manifest committed (provenance), weights gitignored. |
36     | Eval baseline / leaderboard JSON | `eval_baselines/` | Tracked (survives a clean
checkout); the no-regression gate reads it. |
37     | Committed regression fixture (video-free golden) |
`eval_baselines/fixtures/<slug>/` | Tracked, LF-pinned (`gitattributes`). Synthetic Tier-0
(or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with
no video/GPU/DB. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m
backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated). |
38     | Pipeline run outputs (reels, trajectories, DBs, frames) | `output/` | Gitignored.
Per-video isolation is the tracked PER_VIDEO_WORKSPACE.md project (P0 helper shipped #264;
supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`). |
39
40     Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do
not move):
41     `pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) .
`run_all_tests.py` (CI: `ci.yml`) .
42     `run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) .
`run_eval.py`
43     (grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic
`setup.py` moved to
44     `scripts/doctor.py` (it shadowed the build system); invoke via `python
scripts/doctor.py` or `indexer --setup`.
45     New standalone run-drivers go in scripts/`, not the root.
46
47     ---
48
49     ## 1. PIPELINE
50
51     ### 1A. Runtime: video file -> highlight reel
52     ```
53     typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
54
55     @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig

```

```

55         -> video_id = MD5(whole file)                @backend/utils/hashing.py::compute_video_id
(single shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint
copies)
56         -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
57         FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity
-> SceneCut -> Blur -> Relevance (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
58         [validation FAIL -> persist status="failed", return early; report STILL emitted
in finally]
59         -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
60         -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback
'motion')
61         -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)
62         -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
63         [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here
without stitching]
64         -> finally: emit_indexer_report(...) @backend/reporting/__init__.py (? ALWAYS runs
? even on validation failure / exception; NEVER raises; grafts response["reports"])
65         -> select segment ids -> [(start,end)] (+ stitching padding)
66         -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
67         per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
68         ```
69         Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same
shape):

```

32. assistant (2026-07-03T17:35:18.115Z)

Perfect. The CODE_MAP.md clearly states that backend/eval/ is for "Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)". The golden_manifest.py is indeed a library (no `\_\_main\_\_` that side-effects), so the placement is correct. Let me verify it has the proper structure:

33. user (2026-07-03T17:35:19.506Z)

```
377:if __name__ == "__main__":
```

34. assistant (2026-07-03T17:35:21.888Z)

Good! Now let me check if tests/test_golden_manifest.py follows conventions - it should be pure/offline per CODE_MAP:

35. user (2026-07-03T17:35:23.211Z)

(Bash completed with no output)

36. assistant (2026-07-03T17:35:25.834Z)

Good, no gpu/network markers. Now let me check for any issues with doc honesty tags - looking for mismatches in the ALPHA_LAUNCH_READINESS.md:

37. user (2026-07-03T17:35:27.118Z)

```

5a91701 eval: add portable golden manifest check
5eb8755 docs: add alpha launch readiness tracker (#451)
e4914d1 Surface validation failure reasons
f0814d5 Normalize upload filenames before extension checks
e585965 Fix ffmpeg concat clip paths (#449)

```

38. assistant (2026-07-03T17:35:29.456Z)

Now let me carefully check the diff to verify that the docs reference to A2 is correct and is this PR indeed implementing A2:

39. user (2026-07-03T17:35:30.728Z)

5a91701 eval: add portable golden manifest check
6d4514c fix: WASB cache manifest-compat tests on Windows (#353)
81b0cfc fix(wasb): don't abspath the cache key in manifest_compatible (#348) (#350)

40. assistant (2026-07-03T17:35:33.343Z)

Now let me examine the code carefully for any convention violations:

41. user (2026-07-03T17:35:34.702Z)

(Bash completed with no output)

42. assistant (2026-07-03T17:35:37.543Z)

Let me check if all imports are clean:

43. user (2026-07-03T17:35:38.709Z)

```
16:from __future__ import annotations
18:import argparse
19:import json
20:import os
21:import sys
22:from dataclasses import dataclass
23:from pathlib import Path, PureWindowsPath
24:from typing import Any, Iterable, Sequence
```

44. assistant (2026-07-03T17:35:41.933Z)

All stdlib imports, good. Now let me check the imports in the modified backend/eval files to ensure they're clean:

45. user (2026-07-03T17:35:43.107Z)

```
24:from __future__ import annotations
26:import argparse
27:from typing import Callable, Dict, List, Tuple
29:from backend.eval.calibrate_wasb import load_golden_gts
30:from backend.eval.calibration import (
36:from backend.eval.golden_manifest import load_resolved_manifest
37:from backend.pipeline.detectors import rally_gate
38:from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
```

46. assistant (2026-07-03T17:35:45.437Z)

Let me verify that golden_manifest is properly integrated and check for any hidden side effects at import:

47. user (2026-07-03T17:35:46.719Z)

OK

48. assistant (2026-07-03T17:35:49.043Z)

Good, no import-time side effects. Now let me check the test file to make sure it's conventional:

49. user (2026-07-03T17:35:50.247Z)

```

1:import json
2:from pathlib import Path
4:import pytest
6:from backend.eval import golden_manifest as gm

```

50. assistant (2026-07-03T17:35:52.871Z)

Perfect, all standard. Now let me verify that the doc updates match the convention requirements. Let me look at what a proper status update should look like from the ALPHA_LAUNCH_READINESS.md itself:

51. user (2026-07-03T17:35:53.397Z)

```

91      ## 7. Deliverables & progress tracker
92
93      Legend: TODO = not started, IN PROGRESS = actively being worked, DONE =
shipped/verified, GATED = blocked on owner/legal/platform decision. One PR per row unless the
row explicitly names a cross-repo batch.
94
95      | ID | Deliverable | Repo / owner | Depends on | Gated? | Status | PR |
96      |----|-----|-----|-----|-----|-----|----|
97      | A0 | Create this alpha readiness tracker and index it in docs | `badminton-highlight-
indexer` | - | No | DONE | #451 |
98      | A1 | Stand up alpha serving endpoint: Cloud Run/GPU or approved host, health check,
edge auth ON, upload -> process -> jobs -> compile -> download verified | `khelesutra` +
`badminton-highlight-indexer` | A0 | Owner spend / CF config | TODO | - |
99      | A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke
check that fails loudly when labels or trajectories are missing | `badminton-highlight-indexer`
| A0 | No | IN PROGRESS | this PR |
100     | A3 | Promote the 15-video source-video/MD5 records into the collector/vault path;
reconcile collector's six-video source manifest with the 15-video eval corpus | `sports-data-
collector` + vault | A2 | Owner upload/storage scope | TODO | - |
101     | A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
102     | A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6`
floor for future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |
103     | A6 | Fix `backend.eval.ablation` CLI circular import, then run the R2/tolF1 ablation
on the n=15 corpus | `badminton-highlight-indexer` | A2 | No | TODO | - |
104     | A7 | Regenerate or upconvert full-fusion feature JSONs for the newer 9 videos so
fusion/person/audio analyses run over all 15 | `badminton-highlight-indexer` | A2 | GPU/data
availability | TODO | - |
105     | A8 | Re-run `fusion_compare`, group ablation, and served contrast after A7; explicitly
promote/reject person fusion, Gate A served-default, and any other R-series default flips |
`badminton-highlight-indexer` | A6, A7 | Reviewer sign-off | TODO | - |
106     | A9 | Run Gen-0/TemporalMaxer shadow training against the refreshed n=15 floor; produce
a non-serving artifact bundle and gate verdict | `badminton-highlight-indexer` | A5, A8 | Owner
approval for M0.4 scope | TODO | - |
107     | A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum
threshold, significance rule, and what claims are allowed | Owner + `badminton-highlight-
indexer` | A4, A5, A8 | Owner decision | GATED | - |
108     | A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred,
document "no commercial model sharing" copy guardrail | Owner/legal | A10 | Legal/provenance |
GATED | - |
109     | A12 | Reopen or defer P10/P11 corpus flywheel: `PATCH /api/segments`, collector
promotion, `train`/`reeval` job, delta-F1 surface, and alignment with
`COMPUTE_DECOUPLED_SERVING` M11 | `khelesutra` + `badminton-highlight-indexer` + collector | A1,
A3 | Owner D13 reopen (`khelesutra/docs/STUDIO_MEDIA_LIFECYCLE.md`) | GATED | - |
110     | A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure
triage, known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11
| Owner launch decision | TODO | - |

```

52. assistant (2026-07-03T17:35:56.103Z)

Now let me check if the doc honesty tags are consistent and properly placed. Looking at CLAUDE.md again for the convention on doc tagging:

53. user (2026-07-03T17:35:56.609Z)

```
70     ## Workflow conventions
71     - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
72     - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
73     - Never push to the default branch directly; descriptive commit messages.
74     - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
`docs/GOLDEN_REGRESSION_FIXTURES.md`.
75     - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal
state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that
project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR
lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link
breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI
job (`python -m tools.check_md_links`).
76     - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to
the owner,
77     who has the golden corpus + labelled videos; verify pure-logic paths locally.
78
```

54. assistant (2026-07-03T17:35:59.110Z)

Let me check the PROJECT_DOC_TEMPLATE again for honesty-tag conventions:

55. user (2026-07-03T17:35:59.345Z)

```
1      <!--
2      PROJECT DOC TEMPLATE ? copy this file to docs/<YOUR_PLAN>.md and fill it in.
3
4      A "tracked project doc" is how this repo carries substantial design/project work: it can
be
5      ITERATED on, its progress TRACKED, and it is MARKED DONE against explicit deliverables,
then
6      archived. Standardizes the emergent pattern in docs/RALLY_QUALITY_RESEARCH.md §7 and
7      docs/GOLDEN_REGRESSION_FIXTURES.md.
8
9      Rules of thumb:
10     - §7 (the progress tracker) is the SOURCE OF TRUTH ? one row per independently-shippable
PR.
11     - Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs
sign-off),
12       or [design] (proposed). The doc reflects real shipped state, never aspirational.
13     - It POINTS to detail (code anchors, research artifacts), it does not duplicate it.
14     - Sections marked OPTIONAL are included only when relevant (e.g. Foundation/Threat-model
for
15       research- or security/privacy-heavy work). Delete sections that do not apply.
16     - Move to docs/archives/ once Status = DONE (per the archive policy: only terminal-state
work).
17     -->
18
19     # <Project / Plan Title>
20
21     > **Status:** `DRAFT` ? `<owner-gated? | in progress>` . **Owner:** `<name>` .
**Created:** `<YYYY-MM-DD>` . **Last updated:** `<YYYY-MM-DD>`
22     > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
23     > **Tracking anchors:** §7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once work starts).
```

```

24      > **Relation to existing docs:** `<extends | supersedes | peer-of ?>`
25      > **Honesty note:** mark each claim `[verified]` / `[researched]` / `[design]`. Not
legal/financial advice where applicable.
26
27      ---
28
29      ## 0. TL;DR
30      <2?5 sentences: the goal, the approach, and the single most important caveat. Call out
any finding that is already actionable on its own.>

```

56. assistant (2026-07-03T17:36:01.826Z)

Good. Now let me carefully check if there are any violations by examining the diff again. Let me look at the actual status-update row more carefully and compare to how this PR was supposed to work:

57. user (2026-07-03T17:36:03.026Z)

```

| A2 | Make the n=15 golden manifest portable/reproducible; add a path/corpus smoke check that
fails loudly when labels or trajectories are missing | `badminton-highlight-indexer` | A0 | No |
IN PROGRESS | this PR |
| A3 | Promote the 15-video source-video/MD5 records into the collector/vault path; reconcile
collector's six-video source manifest with the 15-video eval corpus | `sports-data-collector` +
vault | A2 | Owner upload/storage scope | TODO | - |
| A4 | Refresh the n=15 nightly regression baseline intentionally and record the exact
command/output; keep stale-baseline warnings until reviewed | `badminton-highlight-indexer` | A2
| Reviewer sign-off | TODO | - |
| A5 | Recompute the heuristic n=15 floor and replace the stale `heuristic_lovo_n6` floor for
future Gen-0 comparisons | `badminton-highlight-indexer` | A2 | No | TODO | - |

```

58. assistant (2026-07-03T17:36:05.750Z)

Perfect. Now let me check the section 8 where smoke commands were added:

59. user (2026-07-03T17:36:06.958Z)

8. Concrete next PR sequence

1. ****Docs PR:**** land this tracker.
2. ****Corpus portability PR:**** rebase/portable manifest tooling plus missing-file smoke test.
3. ****Hosted alpha PR/batch:**** health/auth/upload/process/job/compile/download verification and runbook updates.
4. ****Baseline PR:**** intentional n=15 nightly baseline refresh and heuristic floor refresh.
5. ****Ablation/fusion PRs:**** fix ablation import, complete 15-video features, rerun promotion/rejection decisions.
6. ****Launch decision PR/docs update:**** record ship-gate target, C3 status, alpha copy limits, and launch checklist.

A2 smoke command:

```

```powershell
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count 15
--write output/human_lovo_manifest.local.json
```

```

The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.

9. Open questions

1. ****Owner:**** Is the first alpha allowed to use manual/offline corpus promotion, or should P10/P11 be reopened before any tester wave?

60. assistant (2026-07-03T17:36:09.695Z)

Excellent! Now let me verify there aren't honesty-tag violations in the new section added. The convention from PROJECT_DOC_TEMPLATE is that each claim should be tagged [verified], [researched], or [design]. Let me check if the new smoke command section follows this:

61. user (2026-07-03T17:36:11.025Z)

```
+A2 smoke command:
+
+```powershell
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15
+python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-count
15 --write output/human_lovo_manifest.local.json
+```
+
+The checker resolves stale absolute paths against the repo's sibling `Annotation Setup` folders
and `output/`, then exits non-zero if any required `trajectory` or `labels` file is absent.
+
## 9. Open questions

1. **Owner:** Is the first alpha allowed to use manual/offline corpus promotion, or should
P10/P11 be reopened before any tester wave?
```

62. assistant (2026-07-03T17:36:13.643Z)

Now let me check if the data_pipeline/GOLDEN_VIDEOS.md update has any honesty-tag issues:

63. user (2026-07-03T17:36:14.726Z)

```
diff --git a/docs/data_pipeline/GOLDEN_VIDEOS.md b/docs/data_pipeline/GOLDEN_VIDEOS.md
index 6d68eeb..bb197bd 100644
--- a/docs/data_pipeline/GOLDEN_VIDEOS.md
+++ b/docs/data_pipeline/GOLDEN_VIDEOS.md
@@ -22,6 +22,11 @@ auditable. Updated whenever a new video is labeled + ingested.
- **Ingestion** = copy the trajectory + labels to durable paths, write
`output/<name>_golden_features.json`
  (`gts` + candidates), append a row to `output/human_lovo_manifest.json`. (`scratch/at_par.py`
scores a
  single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole corpus.)
+- **Path smoke check:** before running corpus evals, run
+ `python -m backend.eval.golden_manifest --manifest output/human_lovo_manifest.json --min-
count 15`.
+ The checker rebases stale absolute paths against sibling `Annotation Setup` folders and
`output/`, and
+ exits non-zero if any required trajectory or labels file is missing. Add `--write
output/human_lovo_manifest.local.json`
+ when a resolved local manifest artifact is useful.
- **Guardrails:** owned footage only; **minors excluded**; labels never derived from a paid LLM
(Gemini is
  inference/reference only). Commercial-clean.

@@ -57,4 +62,5 @@ auditable. Updated whenever a new video is labeled + ingested.
## How to isolate the golden videos
Each row's **source proxy** column is the file to copy into a dedicated golden-videos location.
The labels
  live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
-trajectory is in `Collect/Trajectories/` (path in the manifest). Keep this table in sync on
every new ingest.
+trajectory is in `Annotation Setup/Trajectories/` (older manifests may still say
`Collect/Trajectories/`;
+use the path smoke checker above). Keep this table in sync on every new ingest.
```

64. assistant (2026-07-03T17:36:17.370Z)

Now let me run a comprehensive check on the new golden_manifest.py module to look for any other conventions violations. Let me check if it imports anything that creates import-time side effects:

65. user (2026-07-03T17:36:19.629Z)

Exit code 1

Traceback (most recent call last):

```
File "<string>", line 7, in <module>
  spec.loader.exec_module(m)
~~~~~^AAA

File "<frozen importlib._bootstrap_external>", line 1023, in exec_module
File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
File "C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\eval\golden_manifest.py", line 36, in <module>
  @dataclass(frozen=True)
  ~~~~~^AAAAAAAAAAAAAAAA

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\dataclasses.py", line 1295,
in wrap
  return _process_class(cls, init, repr, eq, order, unsafe_hash,
                        frozen, match_args, kw_only, slots,
                        weakref_slot)

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\dataclasses.py", line 997, in
_process_class
  and _is_type(type, cls, dataclasses, dataclasses.KW_ONLY,
  ~~~~~^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
      _is_kw_only))) :
      ^AAAAAAAAAAAAAAAA

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\dataclasses.py", line 757, in
_is_type
  ns = sys.modules.get(cls.__module__).__dict__
      ^AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AttributeError: 'NoneType' object has no attribute '__dict__'. Did you mean: '__dir__'?
```

66. assistant (2026-07-03T17:36:22.107Z)

That's just a quirk of the test approach. Let me check more directly:

67. user (2026-07-03T17:36:23.204Z)

Import successful, no side effects

68. assistant (2026-07-03T17:36:26.648Z)

Good. Now let me check if there are any specific test conventions from CLAUDE.md. Let me look at the 70% coverage floor requirement:

69. user (2026-07-03T17:36:29.060Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 5 items

tests/test_golden_manifest.py::test_resolves_stale_absolute_paths_to_workspace_layout PASSED [ 20%]
tests/test_golden_manifest.py::test_strict_loader_reports_missing_required_files PASSED [ 40%]
tests/test_golden_manifest.py::test_trajectory_only_resolution_leaves_label_optional PASSED [
```

```
60%]
tests/test_golden_manifest.py::test_cli_writes_resolved_manifest PASSED [ 80%]
tests/test_golden_manifest.py::test_cli_fails_loudly_on_missing_file PASSED [100%]
```

```
===== 5 passed in 0.23s =====
```

70. assistant (2026-07-03T17:36:31.520Z)

Good! Tests pass. Now let me check the convention about whether new eval CLI modules should have their imports in a specific location. Let me look at whether there are any issues with the import changes:

71. user (2026-07-03T17:36:32.725Z)

```
diff --git a/backend/eval/calibrate_local.py b/backend/eval/calibrate_local.py
index 687f790..4734f38 100644
--- a/backend/eval/calibrate_local.py
+++ b/backend/eval/calibrate_local.py
@@ -24,7 +24,6 @@ where videos.json = [{"name", "trajectory", "fps", "frame_width", "labels"}, ...]
     from __future__ import annotations

     import argparse
-    import json
     from typing import Callable, Dict, List, Tuple

     from backend.eval.calibrate_wasb import load_golden_gts
@@ -34,6 +33,7 @@ from backend.eval.calibration import (
     leave_one_video_out,
     select_best,
 )
+from backend.eval.golden_manifest import load_resolved_manifest
     from backend.pipeline.detectors import rally_gate
     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

@@ -164,8 +164,7 @@ def main() -> None:
     ap.add_argument("--iou", type=float, default=0.5)
     ap.add_argument("--metric", default="f1", choices=["f1", "precision", "recall"])
     args = ap.parse_args()
-    with open(args.manifest, encoding="utf-8") as f:
-        specs = json.load(f)
+    specs = load_resolved_manifest(args.manifest)
     run(specs, iou=args.iou, metric=args.metric)
```

72. assistant (2026-07-03T17:36:36.177Z)

Good - the removal of the `json` import is appropriate since `load\_resolved\_manifest` handles that. Let me now check if there's any potential issue with the way other modules now call the golden_manifest loader. Let me check test_served_gate_a_regression more carefully:

73. user (2026-07-03T17:36:36.489Z)

```
60     # worktree / CI). Resolve it relative to this repo root first, with an env override for
an
61     # out-of-tree checkout.
62     _REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
63     MANIFEST = os.environ.get(
64         "SERVED_GATE_A_MANIFEST",
65         os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
66     )
67
68
69     def _golden_specs():
70         """Return rebased manifest specs when trajectory + label files are present, else
[]."""
```

```

71         if not os.path.isfile(MANIFEST):
72             return []
73         try:
74             specs = load_resolved_manifest(MANIFEST)
75         except ManifestPathError:
76             return []
77         return specs if len(specs) >= 6 else []
78
79
80     _SPECS = _golden_specs()
81     _skip = pytest.mark.skipif(
82         not _SPECS,
83         reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
84     )
85
86
87     # ---- #398 regression: golden-free (runs in CI; only the golden DATA is absent there,
not cv2) ----
88     def test_served_idx_cfg_is_typed_and_seams_accept_it(tmp_path):
89         """The served litmus must pass a TYPED IndexingConfig (what production passes), not
a dict ?

```

74. assistant (2026-07-03T17:36:39.636Z)

Excellent! The test handles the case where the manifest is absent gracefully. This is good practice. Now let me check if there's a comprehensive review of all the changes. Let me do a final check on all backend/eval files to make sure there are no circular import issues or other problems:

75. user (2026-07-03T17:36:45.728Z)

All imports successful - no circular dependencies

76. assistant (2026-07-03T17:36:48.229Z)

Perfect! Now let me do a final comprehensive review by reading the new golden_manifest module one more time to look for any convention violations I might have missed:

77. user (2026-07-03T17:36:49.433Z)

384

78. assistant (2026-07-03T17:36:52.428Z)

This is a new file with 378 lines. Let me verify the file placement against the CODE_MAP rule one more time by checking if it's importable from other modules (library use):

79. user (2026-07-03T17:36:53.741Z)

```

backend/eval/calibrate_local.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/distill_local.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/fusion_golden.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/rally_seg_eval.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/rally_seq_proto.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/served_gate_a.py:from backend.eval.golden_manifest import load_resolved_manifest
backend/eval/serve_contrast.py:from backend.eval.golden_manifest import load_resolved_manifest
tests/test_golden_manifest.py:from backend.eval import golden_manifest as gm
tests/test_served_gate_a_regression.py:from backend.eval.golden_manifest import ( # noqa: E402
training/gen0/harness.py:from backend.eval.golden_manifest import load_resolved_manifest

```

80. assistant (2026-07-03T17:36:56.780Z)

Perfect! It's being imported widely as a library module (7 backend/eval files + training + tests), which matches the CODE_MAP requirement that "Eval LIBRARY... These are imported widely".

Now let me check if there are any other considerations from the CLAUDE.md workflow conventions I should check:

81. user (2026-07-03T17:36:57.915Z)

1

82. assistant (2026-07-03T17:37:00.109Z)

Good - just one commit. Now let me verify the commit message is descriptive:

83. user (2026-07-03T17:37:01.100Z)

<persisted-output>

Output too large (53.6KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tool-results\bledk6lz0.txt

Preview (first 2KB):

5a91701 eval: add portable golden manifest check
5eb8755 docs: add alpha launch readiness tracker (#451)
e4914d1 Surface validation failure reasons
f0814d5 Normalize upload filenames before extension checks
e585965 Fix ffmpeg concat clip paths (#449)
ac6184b docs: archive the completed 2026-07-02 workspace audit umbrella (#446)
da2d4bf ci: pin ruff (0.15.16) and mypy (2.1.0) in ci + nightly (R1-11/R1-26) (#442)
437c8c7 chore: untrack 7 scratch/*.py scripts to honor the scratch/ convention (R1-24/R1-30) (#441)
204dd00 record final audit cleanup state
c30ff22 finalize audit tracker state
0f32d39 archive roora pilot non-video artifacts (#438)
66e8ceb split database domains behind facade (#437)
d490485 refresh vendored spa bundle (#436)
ea9f0b2 pin obsreport dependency to canonical sha (#435)
14cdc2f pin trackers dependency to immutable sha (#434)
ab2e598 add upload retention scheduler wrapper (#433)
4d8ddb4 refactor(P23 step 2): extract server_runtime.py from main.py + tighten ratchet (#432)
ca22b29 refactor(P23): extract orchestration.py from main.py (god-module split, step 1) + anti-bloat ratchet (#431)
30736ab docs: make AUDIT_REPORT the umbrella code-audit doc; fold + archive the rest (#430)
65983ac docs: reconcile audit docs to shipped state + record PR dispositions (#429)
e0937f3 fix(compute): rescue-or-cancel on Cloud Run bucket-poll timeout (R1-21) (#417)
5613b8d fix(storage): gdrive-api ? chunked streaming download + loss-proof idempotent put (#416)
d00b4db fix gcs storage lazy import typing (#428)
19ebe5a clean remote input scratch dirs (#427)
c6bb4bf bound ffmpeg subprocess timeouts
6c767b5 cache remote source material
9523d17 fix process idempotency cache
e517475 docs: track khelsutra-guru health remediation plan
4f148d1 fix: config-correctness + typing fixes (audit R1-19, R1-20, R1-27) (#422)
d80aa02 docs: truth-sync ? audit report + fix stale state in CLAUDE.md, BACKLOG, DOC_STATUS, README, NEXT_STEPS (#421)
...

</persisted-output>

84. assistant (2026-07-03T17:37:04.042Z)

Perfect - "eval: add portable golden manifest check" is descriptive. Now let me do a comprehensive final check by looking for any pattern of violations. Let me check if there's a docstring convention issue:

85. user (2026-07-03T17:37:05.024Z)

"""Portable golden-corpus manifest loader and smoke checker.

The owner-side golden manifest is intentionally gitignored and has historically carried absolute paths from whichever box ingested a clip. This module keeps that artifact usable after the workspace moves by resolving stale paths against the

86. assistant (2026-07-03T17:37:07.983Z)

Good. Now let me verify that docstrings on functions follow conventions. Let me check one:

87. user (2026-07-03T17:37:08.893Z)

```
def load_manifest(manifest_path: str | os.PathLike[str]) -> list[dict[str, Any]]:
    """Load the flat golden manifest list and validate its outer shape."""

    path = Path(manifest_path)
    with path.open(encoding="utf-8") as f:
        data = json.load(f)
    if not isinstance(data, list):
        raise ValueError(f"golden manifest must be a JSON list: {path}")
    out: list[dict[str, Any]] = []
    for i, row in enumerate(data):
        if not isinstance(row, dict):
            raise ValueError(f"golden manifest row {i} is not an object: {path}")
        out.append(dict(row))
    return out
```

88. assistant (2026-07-03T17:37:11.909Z)

Good - docstrings follow the pattern. Now let me do a comprehensive final scan by running the actual pytest suite to ensure all tests pass:

89. user (2026-07-03T17:39:12.835Z)

Command running in background with ID: b03ip728w. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tasks\b03ip728w.output. You will be notified when it completes. To check interim output, use Read on that file path.

90. assistant (2026-07-03T17:39:14.952Z)

Let me wait for the test to complete:

91. user (2026-07-03T17:39:15.632Z)

<tool_use_error>Blocked: sleep 60 followed by: cat C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\tasks\b03ip728w.output tail -100 echo "Still running or file unavailable". To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run_in_background: true. Do not chain shorter sleeps to work around this block.</tool_use_error>

92. assistant (2026-07-03T17:39:18.931Z)

Let me check the test output file now:

93. user (2026-07-03T17:39:19.457Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

94. assistant (2026-07-03T17:39:29.127Z)

The tests may still be running. Let me do my analysis without waiting - I have reviewed all the conventions and can provide findings now. Let me do one final check on a critical convention

issue - let me verify the CODE_MAP placement once more for the new golden_manifest.py:

Based on my thorough review of the PR against the project conventions defined in CLAUDE.md, CODE_MAP.md, and PROJECT_DOC_TEMPLATE.md, here are my findings:

****Summary of Review:****

- ****File placement (CODE_MAP \$0):**** golden_manifest.py placed in `backend/eval/` ? CORRECT (it's an "Eval LIBRARY ? importable, pure")
- ****Import/side-effects:**** No import-time side effects ? CORRECT
- ****Library usage:**** Imported by 7 backend/eval modules + training + tests ? CORRECT (matches "imported widely")
- ****Test placement:**** tests/test_golden_manifest.py is pure/offline (no GPU/network markers) ? CORRECT
- ****Commit message:**** "eval: add portable golden manifest check" is descriptive ? CORRECT
- ****Documentation:**** ALPHA_LAUNCH_READINESS.md properly updated with smoke commands and A2 status ? CORRECT
- ****Doc honesty tags:**** ALPHA_LAUNCH_READINESS.md section 3 has `[verified]` and `[analysis]` tags per PROJECT_DOC_TEMPLATE \$7 ? CORRECT

****Potential Issue Found:****

Looking at the docs/data_pipeline/GOLDEN_VIDEOS.md section update, there is a missing honesty tag on the new "Path smoke check" bullet point. According to PROJECT_DOC_TEMPLATE.md line 11-12, "Keep it HONEST: tag each claim [verified] (checked against code), [researched] (needs sign-off), or [design] (proposed)."

The new text added to GOLDEN_VIDEOS.md line 25-27 ("Path smoke check:...") makes a claim about the checker's behavior but is not tagged with [verified]/[design].

...

json

```
[
  {
    "file": "docs/data_pipeline/GOLDEN_VIDEOS.md",
    "line": 25,
    "summary": "Missing honesty tag on new claim about golden_manifest checker behavior",
    "failure_scenario": "Per PROJECT_DOC_TEMPLATE $7 and CLAUDE.md conventions,
procedural/operational claims in docs must be tagged [verified]/[design]/[researched]. The new
'Path smoke check' section documents command behavior but lacks such a tag, violating the
honesty convention."
  }
]
```

...